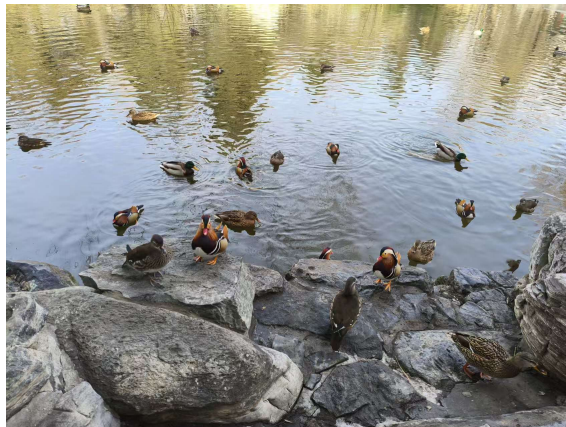


YUANYANG.DSA

Digital Signature Algorithm

Daniel De Almeida Braga
Thomas Espitau
Pierre-Alain Fouque
Huiwen Jia
Paul Kirchner
Bachir Lachguel
Alexandre Wallet
Yang Yu
Shiduo Zhang



June 2026

Abstract. This document specifies the YUANYANG.DSA signature scheme. YUANYANG.DSA is a lattice-based signature scheme following the hash-and-sign paradigm (in the style of Gentry–Peikert–Vaikuntanathan signatures), and instantiated over NTRU lattices. In that sense, it is closely related to, and a successor of, several earlier schemes including Ducas–Lyubashevsky–Prest (DLP), FALCON and MITAKA. More precisely, YUANYANG.DSA offers the “best of both worlds” between FALCON and MITAKA.

FALCON has the advantage of providing short public keys and signatures (offering essentially the best bandwidth trade-off among post-quantum constructions) as well as high security levels; however, it is hampered by a delicate signing algorithm that is difficult to implement correctly, not especially fast for signing, and hard to parallelize. It also has very little flexibility in its parameter settings. In contrast, MITAKA is much simpler to implement, about twice as fast at equal dimension, straightforward to parallelize, and fully versatile in terms of parameters; however, it has lower security than FALCON at equal dimension, has an even more involved key-generation algorithm that tends to be quite slow, and has somewhat larger keys and signatures at equivalent security levels.

YUANYANG.DSA solves the conundrum of choosing between those two schemes by offering all the advantages of both. It uses the same simple, fast, parallelizable signing algorithm as MITAKA, with flexible parameters and without floating-point arithmetic. However, by leveraging the annular NTRU trapdoor-generation approach introduced in ANTRAG, YUANYANG.DSA achieves the same high security and small bandwidth as FALCON. Moreover, it combines recently introduced weak-smoothness Gaussian sampling techniques to further reduce the key and signature sizes. This makes YUANYANG.DSA state of the art for efficient lattice-based signatures over structured lattices.

Keywords: Signature schemes · Lattice-based cryptography · Hash-and-sign paradigm · NTRU lattices · Lattice Gaussian sampling

Table of Contents

1	Introduction.....	1
2	Preliminaries.....	1
3	Design Rationale.....	3
3.1	The GPV hash-and-sign framework.....	3
3.2	Designs of YUANYANG.DSA.....	4
3.3	Criterion for parameter selection.....	4
3.4	Differences with comparable schemes.....	5
3.5	Joint security when used along with YUANYANG.KEM.....	5
4	Algorithm Description.....	5
4.1	Design of KeyGen.....	6
4.1.1	Overview.....	6
4.1.2	Candidate generation: PairGen.....	7
4.1.3	Security-loss filtering: SecurityLoss.....	8
4.1.4	NTRU basis completion: NTRUSolver.....	9
4.1.5	Signing-key precomputation: SamplerPrecomp.....	10
4.2	Design of Sign.....	11
4.2.1	Overview.....	11
4.2.2	FPA-free hybrid preimage sampling: Sample.....	13
4.2.3	Perturbation sampling: Presampler.....	15
4.2.4	Ring Gaussian sampling: RingSamp.....	15
4.3	Design of Verif:.....	16
4.4	Miscellaneous.....	17
4.4.1	Specifications of FFT and FFT^{-1} :.....	17
4.4.2	Integer Gaussian sampling: SamplerZ.....	17
4.4.3	Specifications of Compress and Decompress:.....	20
4.5	List of parameters.....	22
5	Security Statements and Analyses.....	23
5.1	Provable security.....	23
5.2	From provable security to concrete parameters.....	27
5.2.1	Model for lattice reduction.....	28
5.2.2	Key-recovery attack.....	28
5.2.3	Signature forgery by reduction to Approx-CVP.....	28
5.2.4	Concrete security.....	29
5.3	Other attacks on YUANYANG.DSA.....	29
5.3.1	Algebraic attacks.....	29
5.3.2	Overstretched NTRU regime.....	29
5.3.3	Hybrid attacks.....	29
5.4	Joint Security.....	29
6	Performance Evaluation.....	31
6.1	Implementation and test platform.....	31
6.2	Complete API performance.....	32
6.3	Correctness, resources, and storage overhead.....	32
6.4	Cost of the submission auxiliary software.....	33
6.5	Internal performance distribution.....	33
6.6	Batch and online-offline signing.....	33
7	Conclusion.....	34

1 Introduction

Post-quantum signature schemes built from lattices must meet several requirements simultaneously: they should provide compact public keys and signatures, fast signing and verification, convincing concrete security estimates, and an implementation strategy that is realistic on modern platforms. Achieving all of these goals at once is difficult. In the family of hash-and-sign constructions over NTRU lattices, the current landscape is largely shaped by two representative designs. On the one hand, FALCON [PFH⁺22] achieves an excellent bandwidth profile and strong concrete security, but its signing procedure relies on a highly specialised sampler whose implementation is delicate and whose structure lends itself poorly to parallelism and to side-channel countermeasures. On the other hand, MITAKA [EFG⁺22] demonstrates that much simpler signing algorithms are possible, at the cost of a less favourable compactness/security trade-off.

YUANYANG.DSA is designed to reconcile these two lines. It is a GPV-style signature scheme [GPV08] instantiated over NTRU lattices [DLP14], and it aims to retain the compactness and security level of FALCON while using a signing procedure closer in spirit to the simpler approach of MITAKA [EFG⁺22]. Concretely, YUANYANG.DSA uses a modified MITAKAZ sampler that achieves the same Gaussian width as Prest’s hybrid sampler [Pre15] while removing floating-point arithmetic in a simpler and more efficient way. Combined with the annular NTRU trapdoor-generation method of ANTRAG [ENS⁺23], this strips away much of the complexity traditionally associated with signing over structured lattices, without sacrificing the short keys and signatures that make the NTRU-based GPV approach attractive. YUANYANG.DSA also adapts the sampler to the weak-smoothness setting [ZJR⁺25], yielding smaller sizes and stronger security. YUANYANG.DSA offers three concrete parameter sets, in dimensions 512, 1024 and 2048, targeting respectively the 128-bit, 256-bit and 512-bit classical security levels. In particular, all three parameter sets admit a provable-security argument under standard assumptions, namely NTRU and NTRU-ISIS, based on the improved analysis of [JZYT26]. Overall, YUANYANG.DSA is an efficient lattice-based signature scheme over NTRU lattices, achieving *strong security, small size, high speed and good portability*.

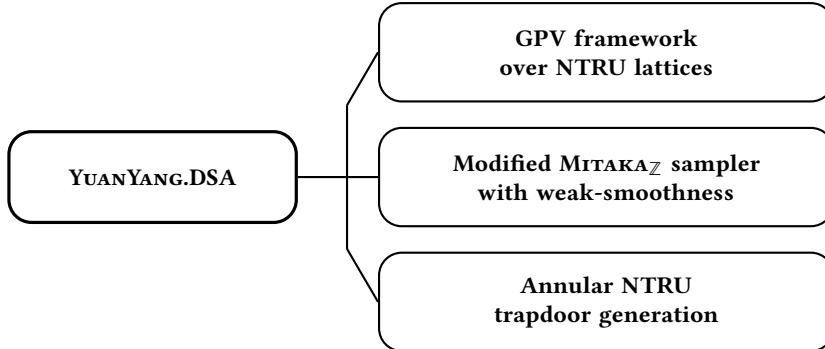


Fig. 1: Main components of YUANYANG.DSA.

Our implementation results back up this positioning. In equal dimension, YUANYANG.DSA has a smaller bandwidth than FALCON, while signing in YUANYANG.DSA is already in the same practical range as in FALCON in our current software, and the online-offline structure leaves room for further speedups (see Section 6.2). Verification has the same algebraic shape as FALCON verification and is expected to benefit from the same implementation techniques. Key generation is currently slower, mainly because the optimized NTRUSolve implementation used in FALCON has not yet been fully adapted to our setting. Nevertheless, it remains practical despite using a different trapdoor-generation strategy. These numbers should be read alongside the analytical discussion later in the document: the concrete security claims rely on standard lattice-cryptanalysis heuristics.

2 Preliminaries

Vectors are written in bold lower case and viewed as column vectors. Matrices are written in bold upper case. When we say that a matrix is a basis of a space, we mean that its columns form the basis. The ℓ_2 -norm of a

vector $\mathbf{x} = (x_1, \dots, x_d)$ is $\|\mathbf{x}\| = (\sum_i |x_i|^2)^{1/2}$, and its ℓ_∞ -norm is $\|\mathbf{x}\|_\infty = \max_i |x_i|$. For a precision parameter p , we denote the rounding of x to precision $1/p$ by $[x]_p := \frac{\lfloor px \rfloor}{p}$.

Lattices. A lattice is a discrete subgroup of $\mathbb{R}^n$, or equivalently the set of *integer* linear combinations of a basis $\mathbf{B}$ of $\mathbb{R}^n$. The volume of a lattice equals $|\det \mathbf{B}|$ for any of its bases.

Power-of-two cyclotomic rings. For $d = 2^n$, we let $K = \mathbb{Q}[X]/(X^d + 1)$ denote the $2d$ -th cyclotomic field. We will mostly work in its ring of integers $R = \mathbb{Z}[X]/(X^d + 1)$, and occasionally in the larger ring $K_{\mathbb{R}} = \mathbb{R}[X]/(X^d + 1)$. Let $\zeta_j = \exp(i(2j-1)\pi/d)$ for $1 \leq j \leq d$ denote the primitive $2d$ -th roots of unity, and, for $f \in K_{\mathbb{R}}$, set $\varphi_i(f) = f(\zeta_i)$. A polynomial in $K_{\mathbb{R}}$ can be described in several ways. The first is the coefficient embedding $f = \sum f_i X^i \mapsto \mathbf{f} = (f_0, \dots, f_{d-1})$. A second is the canonical embedding $f \mapsto \varphi(f) = (\varphi_1(f), \dots, \varphi_d(f))$; the map φ is also known as the Discrete Fourier Transform (DFT), and it sends polynomial multiplication in $K_{\mathbb{R}}$ to coordinate-wise multiplication. We write $K_{\mathbb{R}}^{++}$ for the set of $f \in K_{\mathbb{R}}$ whose canonical coordinates satisfy $\varphi_i(f) \in \mathbb{R}_+$ for all i , and we have $\|\varphi(f)\| = \sqrt{d} \|\mathbf{f}\|$. For $f \in K_{\mathbb{R}}$ we let f^* denote its complex conjugate; explicitly,

$$f^* = f_0 - f_{d-1}X - \dots - f_1 X^{d-1}, \quad \varphi(f^*) = (\overline{\varphi_i(f)})_i.$$

A third representation of cyclotomic elements is via the multiplication matrix

$$f \mapsto [f] := \begin{bmatrix} f_0 & -f_{d-1} & \dots & -f_1 \\ f_1 & f_0 & \dots & -f_2 \\ \vdots & & \ddots & \vdots \\ f_{d-1} & f_{d-2} & \dots & f_0 \end{bmatrix}.$$

One checks that $[f^*] = [f]^t$. In algorithms, the notation $(f_i)_i \in \mathbb{R}^d$ emphasises that the coefficient vector is being manipulated, whereas $f \in K_{\mathbb{R}}$ implicitly indicates the Fourier representation.

Gram–Schmidt orthogonalization over rings. For $(f, g), (F, G) \in K_{\mathbb{R}}^2$, define

$$\langle (f, g), (F, G) \rangle_K = f^* F + g^* G.$$

The Gram–Schmidt orthogonalization of $(F, G) \in K_{\mathbb{R}}^2$ with respect to (f, g) is

$$(\tilde{F}, \tilde{G}) = (F, G) - \frac{\langle (f, g), (F, G) \rangle_K}{\langle (f, g), (f, g) \rangle_K} \cdot (f, g),$$

and one checks that $\langle (f, g), (\tilde{F}, \tilde{G}) \rangle_K = 0$.

NTRU. Let q be a positive integer and let $R_q = R/qR$. We write $f \in R_q^\times$ if the reduction of $f \in R$ modulo q is invertible in R_q ; equivalently, $\gcd(\det[f], q) = 1$. Set $h = g/f \bmod q$ and consider the NTRU module associated to h ,

$$\mathcal{M}_{\text{NTRU}} = \{(u, v) \in R^2 : hu - v \equiv 0 \pmod{q}\},$$

together with its lattice incarnation

$$\mathcal{L}_{\text{NTRU}} = \{(\mathbf{u}, \mathbf{v}) \in \mathbb{Z}^{2d} : [h]\mathbf{u} - \mathbf{v} \equiv 0 \pmod{q}\}.$$

This lattice has volume q^d . Over R , it is generated by (f, g) together with any (F, G) satisfying $fG - gF = q$. For such a pair, $\mathcal{L}_{\text{NTRU}}$ admits a basis of the form

$$\mathbf{B}_{f,g} = \begin{bmatrix} [f] & [F] \\ [g] & [G] \end{bmatrix},$$

and one checks that $([h], -\mathbf{I}_d) \cdot \mathbf{B}_{f,g} \equiv 0 \pmod{q}$, so the verification key is simply h ; further details are given in Section 4.1. The NTRU search problem asks, given $h = g/f \bmod q$, to recover a short pair functionally equivalent to (f, g) , for instance a unit multiple $(f', g') = (\mu f, \mu g)$. Its decision variant asks instead to distinguish $h = g/f \bmod q$ from a uniformly random $h \in R_q := \mathbb{Z}[X]/(q, X^d + 1) = (\mathbb{Z}/q\mathbb{Z})[X]/(X^d + 1)$. Both problems are conjectured to be intractable for sufficiently large d .

Quality of an NTRU basis. The secret basis cannot be arbitrary: it must also enable the sampling of short Gaussian vectors in $\mathcal{L}_{\text{NTRU}}$ via the hybrid sampler. We now define the *quality* of an NTRU basis $\mathbf{B}_{f,g}$:

$$\mathcal{Q}(f, g) = \max_{1 \leq i \leq d/2} \max \left(\frac{|\varphi_i(f)|^2 + |\varphi_i(g)|^2}{q}, \frac{q}{|\varphi_i(f)|^2 + |\varphi_i(g)|^2} \right)^{1/2}.$$

Only half of the embeddings appear, since the remaining ones are obtained by complex conjugation. This is one of the main parameters driving the security of the scheme: $\mathcal{Q}(f, g)$ is always at least 1, and the closer it is to 1, the harder forgery becomes.

Gaussians. The Gaussian function centred at $\mathbf{c} \in \mathbb{R}^d$ with positive-definite covariance Σ is $\rho_{\mathbf{c}, \Sigma}(\mathbf{x}) = \exp(-\frac{1}{2}(\mathbf{x} - \mathbf{c})^T \Sigma^{-1}(\mathbf{x} - \mathbf{c}))$. The normal distribution $\mathcal{N}_{\mathbf{c}, \Sigma}$ has density proportional to $\rho_{\mathbf{c}, \Sigma}$. When we write $x \leftarrow \mathcal{N}_{\Sigma}^{K_{\mathbb{R}}}$, we mean that the corresponding d -dimensional real vector

$$\sqrt{\frac{2}{d}}(\Re \varphi_1(x), \Im \varphi_1(x), \dots, \Re \varphi_{d/2}(x), \Im \varphi_{d/2}(x))$$

follows $\mathcal{N}_{\Sigma}$, where $\Re z$ and $\Im z$ denote the real and imaginary parts of $z \in \mathbb{C}$. For a lattice $\mathcal{L} \subset \mathbb{R}^d$, the discrete Gaussian distribution over $\mathcal{L}$ with parameters $\mathbf{c} \in \mathbb{R}^d$ and Σ is defined, for all $\mathbf{x} \in \mathcal{L}$, by

$$D_{\mathcal{L}, \mathbf{c}, \Sigma}(\mathbf{x}) = \frac{\rho_{\mathbf{c}, \Sigma}(\mathbf{x})}{\rho_{\Sigma}(\mathcal{L} - \mathbf{c})}.$$

We omit the centre when it is 0, and when $\Sigma = s^2 \mathbf{I}$ is scalar we write $\mathcal{N}_s$ or $D_{\mathcal{L}, \mathbf{t}, s}$.

3 Design Rationale

The leading design principles of YUANYANG.DSA include **small bandwidth** (as measured in combined public key and signature size), **fast signing and verification** as well as **solid and well-understood security**. This led us to consider lattice-based signatures following the GPV hash-and-sign paradigm [GPV08]. The GPV hash-and-sign scheme comes with a modular security proof in the random oracle model. When instantiated over structured lattices, GPV lattice-based signatures can offer fast signing and verification (in quasi-linear time) as well as small key and signature sizes.

FALCON is the representative GPV signature algorithm selected by the US NIST for post-quantum standardization. FALCON has an excellent bandwidth profile among post-quantum signatures, fast signing and verification, and strong security. However, it has a highly complicated implementation and relies heavily on floating-point arithmetic. To reduce these drawbacks, MITAKA [EFG⁺22] was proposed as a simplified variant of FALCON using a simpler and parallelizable Gaussian sampler, at the cost of some security loss. Later, Espitau et al. presented an improved NTRU trapdoor-generation approach [ENS⁺23], making MITAKA as secure as FALCON; this technique was then used by the SOLMAE signature scheme [KTW⁺24], which has become a Korean TTA standard.

YUANYANG.DSA is an efficient GPV hash-and-sign instantiation over NTRU lattices. Its security is based on standard NTRU lattice assumptions, namely NTRU and NTRU-ISIS, which have been studied for more than 20 years [CS97, GS02]. By leveraging several recent techniques [ZJR⁺25, JZYT26], YUANYANG.DSA offers the “best of both worlds” between FALCON and MITAKA: it preserves the compact NTRU-based GPV interface of the former while shifting the signing procedure toward the simpler, more implementation-friendly style of the latter.

3.1 The GPV hash-and-sign framework

In a signature scheme, three algorithms must be instantiated: **KeyGen**, **Sign** and **Verif**. The GPV framework adds the following two requirements:

- a class of lattices on which **KeyGen** can compute trapdoor pairs;
- a procedure **Sample**, used by **Sign**, that generates random vectors in these lattices following a distribution that does not leak information about the trapdoor.

Concretely, **KeyGen** outputs a trapdoor pair $(\mathbf{A}, \mathbf{B})$ for a lattice $\mathcal{L}$ satisfying $\mathbf{AB} = 0 \bmod q$, where q is a public integer; the lattice $\mathcal{L}$ is then spanned by $\mathbf{B}$. Signing and verification proceed as follows:

1. **Sign**($\mathbf{B}, M$) first applies a cryptographic hash function H to obtain a vector $\mathbf{m} = H(M)$ in the ambient space $\mathbb{R}^m$, and computes a preimage $\mathbf{c} = \mathbf{A}^{-1}\mathbf{m}$. The subroutine **Sample**($\mathbf{B}, \mathbf{c}$) then returns a random vector $\mathbf{v} \in \mathcal{L}$ close to $\mathbf{c}$. Knowledge of the trapdoor $\mathbf{B}$ ensures that $\mathbf{s} = \mathbf{c} - \mathbf{v}$ is short.
2. **Verif**($\mathbf{A}, M, \mathbf{s}, \gamma$) recomputes $\mathbf{m} = H(M)$ and $\mathbf{c}' = \mathbf{A}\mathbf{s} \bmod q$. If $\mathbf{s}$ is valid, it can be written as $\mathbf{s} = \mathbf{c} - \mathbf{v}$ with $\mathbf{v} \in \mathcal{L}$, so $\mathbf{c}' = \mathbf{A}\mathbf{c} = \mathbf{m} \bmod q$; **Verif** therefore checks this identity first. Since a valid signature must also be short, **Verif** additionally checks that $\|\mathbf{s}\|$ does not exceed a fixed public bound γ . The signature is accepted if and only if both checks succeed.

Beyond correctness, the resulting scheme must be *efficient* in practice: it should consume as little bandwidth as possible while keeping signing and verification fast. Concretely, this means that:

- **KeyGen** should produce *compact* trapdoor pairs, i.e. pairs with a small bit-representation;
- **Sample** should be fast and output short vectors; the shortness depends on the *quality* of the trapdoor;
- **Sign** should encode these random vectors into the smallest possible signature.

Intuitively, the shorter the output of **Sample**, the harder it becomes for an attacker to forge a valid signature; conversely, only a sufficiently good trapdoor allows **Sample** to return short vectors at all. The technical subtleties begin once one realises that the very notion of a “good” trapdoor depends heavily on how **Sample** is instantiated. Optimising this interplay is the main guiding principle of our design, and indeed of GPV instantiations in general.

3.2 Designs of YUANYANG.DSA

The main technical choice in YUANYANG.DSA is to clearly separate the two tasks that are often entangled in earlier designs:

1. **Trapdoor Generation**: generating a structured NTRU basis of the right geometric quality;
2. **Trapdoor Sampling**: exploiting the trapdoor basis through a Gaussian sampler that remains simple, both mathematically and at the implementation level.

The first task is handled by the annular NTRU trapdoor-generation procedure of ANTRAG [ENS⁺23]. The algorithm samples the first basis vector directly in Fourier representation and then rounds its coefficients to integers so as to meet a target quality parameter. Then it completes the NTRU basis with the `NtruSolve` machinery of Prest and Pornin [PP19]. While this introduces additional algebraic structures to the key pairs, no weakness was found by thorough cryptanalysis for concrete parameters.

The second task is handled by a modified MITAKAZ sampler that avoids the algorithmic intricacy of Fast Fourier sampling [DP16] while remaining efficient and naturally amenable to parallel execution. Compared to the original MITAKAZ sampler, our modified sampler removes floating-point arithmetic in a simple and more efficient way. Specifically, it performs Gaussian sampling with an approximate square Gram root decomposition to avoid the overly wide integral Gram decomposition [DGPY20] and then it rectifies the discrepancy with rejection sampling. In addition, we adapt the sampler to the weak-smoothness setting [ZJR⁺25]. This allows us to work with a larger closeness parameter in the smoothing analysis, and thus gives smaller sizes and higher concrete security.

3.3 Criterion for parameter selection

YUANYANG.DSA adopts a conservative criterion for parameter selection.

1. **Solid theoretical security**: YUANYANG.DSA selects parameters that admit a provable-security argument under standard assumptions, based on the improved analysis of [JZYT26]. In fact, the weak-smoothness GPV framework [ZJR⁺25] allows for smaller parameters if an additional non-standard assumption is used. However, YUANYANG.DSA insists on a well-studied security foundation at the cost of some efficiency loss.
2. **Conservative security estimate**: The concrete security of YUANYANG.DSA is estimated as the Core-SVP hardness of best known attacks. Under this conservative model, three parameter sets of YUANYANG.DSA can achieve 128/80-bit, 256/128-bit and 512/256-bit classical/quantum security respectively. In addition, YUANYANG.DSA chooses parameters such that the maximum signature norm remains strictly smaller than the modulus. This requires a slightly larger modulus, but rules out the so-called small- q attacks [DEP23].

3.4 Differences with comparable schemes

The design of YUANYANG.DSA is indeed deeply inspired by the evolution of FALCON-type signatures. Its sampler is a modification of the MITAKAZ sampler, achieving almost identical quality. Also, YUANYANG.DSA reuses the annular NTRU trapdoor generation of ANTRAG. Based on these, it further leverages some new techniques to obtain improved security, efficiency and portability.

More concretely, YUANYANG.DSA adopts the weak-smoothness GPV preimage sampling technique [ZJR⁺25]. This reduces the signature length, and further allows a smaller modulus q while keeping the signature norm bound not exceed q . Instead of pursuing extreme performance with non-standard assumptions, YUANYANG.DSA chooses moderate parameters offering provable security based on standard assumptions via improved GPV analysis [JZYT26]. Furthermore, it removes floating-point arithmetic in a more efficient way than MITAKAZ, which requires much less integer Gaussian samplings.

By combining these techniques, YUANYANG.DSA improves on comparable schemes in terms of security, performance and portability.

Security. YUANYANG.DSA uses the same hardness assumptions as FALCON and MITAKA, but its three parameter sets can achieve 128-bit, 256-bit and 512-bit classical security using power-of-two cyclotomic rings of degrees 512, 1024 and 2048 for up to 2^{80} signatures. Note that FALCON, MITAKA and their variants cannot achieve 128-bit security in equal dimension, even for 2^{64} signatures.

Performance. YUANYANG.DSA has smaller key and signature sizes than FALCON and SOLMAE that is a Korean TTA standard based on MITAKA and ANTRAG. In addition, both signing and verification of YUANYANG.DSA are practically fast. This makes YUANYANG.DSA an attractive drop-in general-purpose signature scheme.

Portability. Compared to FALCON, the signing procedure of YUANYANG.DSA is substantially simplified. In addition, another primary advantage of YUANYANG.DSA over other FALCON-type signatures is its simple and efficient FPA-free sampler. This eases secure implementation and improves performance on constrained hardware platforms without floating-point units.

3.5 Joint security when used along with YUANYANG.KEM

In addition to standalone signatures, the DSA component is paired with YUANYANG.KEM. The two primitives are designed to use the same long-term key pair. This is motivated by applications where authentication and key establishment are both needed, but maintaining two independent public keys increases certificate size, key-distribution cost and secret-key storage. This is of clear interest in some real-world scenarios, e.g. chip-based payment cards [DLP⁺12], end-to-end-encrypted messaging [FGPZ25] and constrained environments where key management is as important as the cost of the cryptographic operations themselves.

However, using the same key for two primitives is not a consequence of their standalone security. A signing oracle and a decapsulation oracle give the adversary different views of the same secret material, and their interaction must be analysed explicitly. For this reason, YUANYANG.DSA is not proposed as a generic key-reuse mechanism with arbitrary KEMs. It is designed to be paired with YUANYANG.KEM, and their same-key use is included in the security analysis.

The compatibility comes from the common NTRU key structure. Both components use the same public key $h = g/f \bmod q$ and the same short secret (f, g) . The DSA component further completes the NTRU basis for trapdoor sampling. The KEM component uses the same secret material together with its own auxiliary values for decapsulation and implicit rejection. The formal joint-security statement is given in Section 5.4. There, YUANYANG.DSA is analysed with access to a decapsulation oracle for YUANYANG.KEM, and YUANYANG.KEM is analysed with access to a signing oracle for YUANYANG.DSA.

4 Algorithm Description

This section describes the YUANYANG.DSA signature scheme and explains how we instantiate the triple **KeyGen**, **Sign**, **Verif**. The scheme is specified by the following parameters:

- the ring $R = \mathbb{Z}[X]/(X^d + 1)$ and the NTRU modulus q ;

- the standard deviation of the integer Gaussians, denoted by η , which is the smoothing parameter of $\mathbb{Z}$ with respect to the closeness parameter ϵ ;
- the standard deviation of the Gaussian corresponding to signatures, denoted by σ_{sig} , and the signature norm bound $\gamma = \tau \cdot \sigma_{\text{sig}} \cdot \sqrt{2d}$, where τ is the slack factor listed in Table 3;
- the quality parameter $\alpha > 1$ and two radii parameters (R_+, R_-) for the annular NTRU trapdoor generation.

Concrete parameters are summarized in Table 3 at the end.

4.1 Design of **KeyGen**

4.1.1 Overview

Key generation computes a short NTRU basis

$$\mathbf{B} = [\mathbf{b}_1 \mid \mathbf{b}_2] = [(f, g) \mid (F, G)]$$

as the signing key; the associated public key is $h = g/f \bmod q$. The procedure has four main steps, implemented by the following subroutines.

1. The **PairGen** algorithm generates a candidate pair (f, g) using the annular NTRU trapdoor-generation method [ENS⁺23]. It samples the Fourier representations of f and g within a prescribed annular region, rounds the resulting real polynomials to elements of R , and returns a pair satisfying the target quality bound.
2. The **SecurityLoss** algorithm estimates the candidate-dependent security loss arising from the random-oracle simulation [JZYT26]. Together with the norm and invertibility checks performed by **KeyGen**, it filters unsuitable candidates before the more expensive basis-completion step.
3. The **NTRUSolver** algorithm is the recursive basis-completion procedure of Prest and Pornin [PP19]. Given an accepted pair (f, g) , it computes (F, G) satisfying

$$fG - gF = q \pmod{X^d + 1},$$

and hence produces a basis $\mathbf{B} = [(f, g) \mid (F, G)]$ of the corresponding NTRU lattice. If the completion fails, **KeyGen** restarts with a new candidate.

4. The **SamplerPrecomp** algorithm derives the basis-dependent data required by **Sign** and **Sample**. In particular, it computes the rounded Gram–Schmidt coefficient $\hat{u}$, the triangular inverse $\hat{\mathbf{B}}^{-1}$, the rounded covariance factor $\hat{\mathbf{A}}$, the residual covariance Σ_δ , and the constants r and C . These quantities are computed once during **KeyGen** and reused for all subsequent signatures.

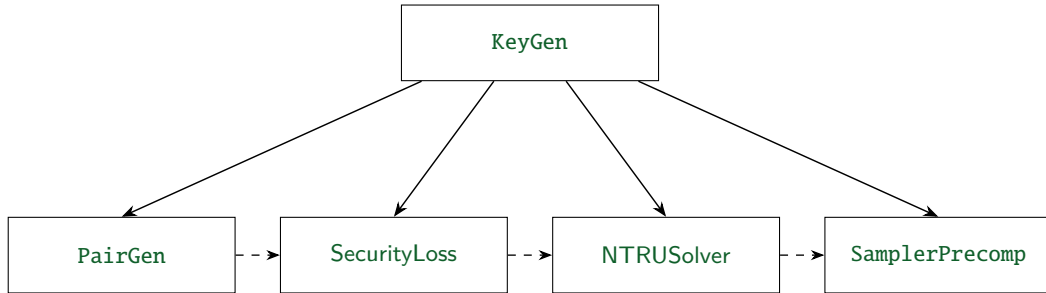


Fig. 2: The flowchart of **KeyGen**.

Algorithm 1 KeyGen	alg:keygen
Input The scheme parameters, including a modulus q, a target quality parameter $\alpha > 1$, and parameters $\sigma_{\text{sig}} = \eta \cdot \sigma$ with $\eta > 0$. Output A secret key sk and a public key pk.	
<pre> 1. // Secret basis generation and filtering 2. repeat 3. repeat 4. $\mathbf{b}_1 := (f, g) \leftarrow \text{PairGen}(q, \alpha, R_-, R_+)$ 5. $\text{loss} \leftarrow \text{SecurityLoss}((f, g))$ 6. until $\ (f, g)\ ^2 \geq \text{Norm}_{\text{fg}} \wedge \text{loss} \leq \text{LossBound} \wedge f \in R_2^\times \wedge g \in R_q^\times$ 7. $\mathbf{b}_2 := (F, G) \leftarrow \text{NTRUSolver}(q, f, g)$ 8. until $\mathbf{b}_2 \neq \perp$ 9. $\mathbf{B} \leftarrow [\mathbf{b}_1 \mid \mathbf{b}_2]$ 10. // Public-key data computation 11. $h \leftarrow g/f \bmod q$ ▷ signature norm bound 12. // Signing-key precomputation 13. $(\hat{u}, \hat{\mathbf{B}}^{-1}, \hat{\mathbf{A}}, \Sigma_\delta, r, C) \leftarrow \text{SamplerPrecomp}(\mathbf{B}, \sigma, \eta, p_1, p_2)$ 14. $\text{sk} \leftarrow (\mathbf{B}, \hat{u}, \hat{\mathbf{B}}^{-1}, \hat{\mathbf{A}}, \Sigma_\delta, r, C)$ 15. $\text{pk} \leftarrow (q, h, \sigma_{\text{sig}}, \eta, \gamma)$ 16. return sk, pk </pre>	

4.1.2 Candidate generation: **PairGen**

This algorithm is the annular trapdoor-generation step introduced in ANTRAG [ENS⁺23], specialised to our notation. The NTRU module associated to a public key h is morally a 2-dimensional object: when q is the lattice modulus and $((f, g), (F, G))$ is a basis, we have

$$\det(\mathcal{M}_{\text{NTRU}})^2 = (f f^\star + g g^\star)(\tilde{F} \tilde{F}^\star + \tilde{G} \tilde{G}^\star) = q^2 \in R,$$

just as the area of a rectangle is the product of its sides. Consequently, the value of $\tilde{F} \tilde{F}^\star + \tilde{G} \tilde{G}^\star$ is fully determined by (f, g) and q , which is why *only* (f, g) is needed in order to predict the quality of the basis produced by **NTRUSolver**. Recall from Section 2 that this quality is

$$\alpha := \mathcal{Q}(f, g) = \max_{1 \leq i \leq d/2} \max \left(\frac{|\varphi_i(f)|^2 + |\varphi_i(g)|^2}{q}, \frac{q}{|\varphi_i(f)|^2 + |\varphi_i(g)|^2} \right)^{1/2}.$$

Equivalently, a basis has quality α if and only if, for every $i \leq d/2$,

$$\frac{q}{\alpha^2} \leq |\varphi_i(f)|^2 + |\varphi_i(g)|^2 \leq \alpha^2 q. \quad (1)$$

Since this condition depends only on the evaluations of f and g — equivalently, on their FFT representations — it is natural to sample these polynomials directly through their complex evaluations. This is handled by the subroutine **UniFCrown**, described in Section 4.4 (Algorithm 3).

One subtlety is that inverting the **FFT** a priori yields only real polynomials $f^{\mathbb{R}}, g^{\mathbb{R}} \in K_{\mathbb{R}}$. We therefore round their coefficients to the nearest integers, which may move the resulting pair slightly away from the ideal annular region.

To leave room for this rounding error, we sample the Fourier magnitudes from a slightly narrower annulus. More precisely, we take the radii R_- and R_+ to be the following complementary convex combinations of $\alpha\sqrt{q}$ and $\sqrt{q}/\alpha$:

$$R_- = \left(\frac{1}{3} \alpha + \frac{2}{3\alpha} \right) \sqrt{q}, \quad R_+ = \left(\frac{2}{3} \alpha + \frac{1}{3\alpha} \right) \sqrt{q}.$$

This choice should be understood as an experimentally calibrated margin around the ideal quality interval. It gives good acceptance rates for the parameter sets considered here, while the final quality bound is still checked after rounding. We use the **PairGen** procedure below with default target qualities $\alpha_{512} = 1.29$, $\alpha_{1024} = 1.4$, and $\alpha_{2048} = 1.43$.

Algorithm 2 PairGen		alg:pairgen
Input	A modulus q , a target quality parameter $1 < \alpha$, two radii parameters $R_+ > R_- > 0$.	
Output	A pair (f, g) with $\mathcal{Q}(f, g) \leq \alpha$.	
1.	for $i = 1$ to $d/2$ do	
2.	$x_i, y_i \leftarrow \text{UnifCrown}(R_-, R_+)$	$\triangleright$ see Algorithm 3
3.	$\theta_x, \theta_y \leftarrow \mathcal{U}(0, 1)$	
4.	$\varphi_{f,i} \leftarrow x_i \cdot e^{2i\pi\theta_x}$	
5.	$\varphi_{g,i} \leftarrow y_i \cdot e^{2i\pi\theta_y}$	
6.	$(f^{\mathbb{R}}, g^{\mathbb{R}}) \leftarrow (\text{FFT}^{-1}((\varphi_{f,i})_{i \leq d/2}), \text{FFT}^{-1}((\varphi_{g,i})_{i \leq d/2}))$	
7.	$(\mathbf{f}, \mathbf{g}) \leftarrow ((\lfloor f_i^{\mathbb{R}} \rfloor)_{0 \leq i < d}, (\lfloor g_i^{\mathbb{R}} \rfloor)_{0 \leq i < d})$	
8.	$(\varphi(f), \varphi(g)) \leftarrow (\text{FFT}(\mathbf{f}), \text{FFT}(\mathbf{g}))$	
9.	for $i = 1$ to $d/2$ do	
10.	if $q/\alpha^2 > \varphi_i(f) ^2 + \varphi_i(g) ^2$ or $\alpha^2 q < \varphi_i(f) ^2 + \varphi_i(g) ^2$ then restart	
11.	return $(\mathbf{f}, \mathbf{g})$	

Annulus sampling. Algorithm 3 returns a uniformly distributed point in the planar annulus

$$A_+(R_-, R_+) = \{(x, y) \in \mathbb{R}_{\geq 0}^2 : R_-^2 \leq x^2 + y^2 \leq R_+^2\}$$

starting from uniform random samples in $(0, 1)$ — meaning uniformly distributed over the floating-point numbers in $(0, 1)$ with the given mantissa and exponent.

Algorithm 3 UnifCrown		alg:crown
Input	The radii parameters $R_+ > R_- > 0$.	
Output	A point (x, y) with uniform distribution in $A_+(R_-, R_+)$.	
<pre>1. $u_\rho, u_\theta \leftarrow \mathcal{U}(0, 1)$ 2. $\rho \leftarrow \sqrt{R_-^2 + u_\rho(R_+^2 - R_-^2)}$ 3. $x \leftarrow \rho \cdot \cos(\frac{\pi}{2} u_\theta)$ 4. $y \leftarrow \rho \cdot \sin(\frac{\pi}{2} u_\theta)$ 5. return (x, y)</pre>		

4.1.3 Security-loss filtering: **SecurityLoss**

Algorithm 4 estimates the candidate-dependent security loss arising from the random-oracle simulation using the basis-specific analysis of Jia et al. [JZYT26]. It is invoked by **KeyGen** after **PairGen** and before **NTRUSolver**. A candidate is retained only if the resulting estimate does not exceed LossBound.

At a high level, the algorithm first computes the GSO profile $(\ell_0, \dots, \ell_{2d-1})$ of $\mathbf{B}$ by the fFLDL algorithm of FALCON.

$$\ell_i = \|\tilde{b}_i\|^2 \text{ for } i < d$$

$$\ell_{2d-1-i} = q^2 / \ell_i \text{ for } 0 \leq i < d.$$

As per the arguments of [JZYT26, Theorem 3 and Lemma 9], the algorithm computes $\bar{\epsilon}$ such that

$$\sigma_{\text{sig}} = \max_i \sqrt{\ell_i} \cdot \sqrt{\frac{\log(4d(1+1/\bar{\epsilon}))}{2}} \cdot \frac{1}{\pi}$$

and

$$\epsilon' = \frac{T(\bar{\epsilon})}{1 - T(\bar{\epsilon})}$$

where $T(\bar{\epsilon}) = \sum_{i=0}^{2d-1} 2 \left(\frac{\bar{\epsilon}}{4d(1+\bar{\epsilon})} \right)^{\gamma_i}$ with $L_{\max} = \max_j \ell_j$ and $\gamma_i = 2L_{\max}/\ell_i$. Then the resulting profile-based loss is

$$\text{ProfileLoss}(\{\ell_i\}, \epsilon') = \min_{a>1} \left\{ \frac{\lambda}{a} + C_s \log_2 \left(1 + \frac{a\epsilon'}{2} \right) \right\}$$

where C_s is the repetition budget for the parameter set. The pair (f, g) is accepted only if

$$\text{ProfileLoss}(\{\ell_i\}, \epsilon') \leq \text{LossBound}.$$

Algorithm 4 SecurityLoss		alg:securityloss
Input	The NTRU secret key (f, g) .	
Output	An estimated RO-simulation bit loss loss for the current pair (f, g) .	
1.	$(\ell_0, \dots, \ell_{d-1}) \leftarrow (\ \tilde{b}_0\ ^2, \dots, \ \tilde{b}_{d-1}\ ^2)$	$\triangleright$ by <i>ffLDL of FALCON</i>
2.	$(\ell_d, \dots, \ell_{2d-1}) \leftarrow (q^2/\ \tilde{b}_{d-1}\ ^2, \dots, q^2/\ \tilde{b}_0\ ^2)$	$\triangleright$ by <i>NTRU symplecticity</i>
3.	compute $\bar{\epsilon}$ such that $\sigma_{\text{sig}} = \max_i \sqrt{\ell_i} \cdot \sqrt{\frac{\log(4d(1+1/\bar{\epsilon}))}{2}} \cdot \frac{1}{\pi}$	
4.	$L_{\max} \leftarrow \max_j \ell_j$, $\gamma_i \leftarrow \frac{2L_{\max}}{\ell_i}$, $T(\bar{\epsilon}) \leftarrow \sum_{i=0}^{2d-1} 2 \left(\frac{\bar{\epsilon}}{4d(1+\bar{\epsilon})} \right)^{\gamma_i}$	
5.	$\epsilon' \leftarrow \frac{T(\bar{\epsilon})}{1-T(\bar{\epsilon})}$	
6.	loss $\leftarrow \text{ProfileLoss}(\{\ell_i\}, \epsilon')$	
7.	return loss	

4.1.4 NTRU basis completion: **NTRUSolver**

Algorithm 5 is the recursive NTRU basis-completion procedure of Prest and Pornin [PP19]. Given an accepted pair $(f, g) \in R^2$, it returns polynomials $(F, G) \in R^2$ satisfying

$$fG - gF = q \pmod{X^d + 1},$$

or $\perp$ if no completion is obtained.

The algorithm recursively applies the field norm to reduce the ring dimension, lifts the solution to the original ring, and invokes **Reduce** to shorten the resulting second basis vector. If the algorithm returns $\perp$, **KeyGen** discards the current candidate and starts again.

Algorithm 5 NTRUSolver _{d,q}		alg:NTRUSolver
Input $f, g \in \mathbb{Z}[X]/(X^d + 1)$ with d a power of two. Output Polynomials F, G such that $fG - gF = q \bmod X^d + 1$.		
<pre> 1. if $d = 1$ then 2. Compute $u, v \in \mathbb{Z}$ such that $uf - vg = \gcd(f, g)$ ▷ using the extended GCD 3. if $\gcd(f, g) \neq 1$ then return $\perp$ 4. $(F, G) \leftarrow (vq, uq)$ 5. return (F, G) 6. else 7. $f' \leftarrow f(x) \cdot f(-x)$ ▷ $f', g', F', G' \in \mathbb{Z}[X]/(X^{d/2} + 1)$ 8. $g' \leftarrow g(x) \cdot g(-x)$ 9. $(F', G') \leftarrow \text{NTRUSolver}_{d/2,q}(f', g')$ ▷ recursive call 10. $F \leftarrow F'(x^2)g(-x)$ ▷ $F, G \in \mathbb{Z}[X]/(X^d + 1)$ 11. $G \leftarrow G'(x^2)f(-x)$ 12. $(F, G) \leftarrow \text{Reduce}(f, g, F, G)$ ▷ (F, G) is reduced w.r.t. (f, g) 13. return (F, G) </pre>		

Algorithm 6 Reduce		alg:Reduce
Input Polynomials $f, g, F, G \in \mathcal{R}$. Output (F, G) reduced with respect to (f, g) .		
<pre> 1. repeat 2. $k \leftarrow \left\lfloor \frac{Ff^* + Gg^*}{ff^* + gg^*} \right\rfloor$ 3. $F \leftarrow F - kf$ 4. $G \leftarrow G - kg$ 5. until $k = 0$ 6. return (F, G) </pre>		

4.1.5 Signing-key precomputation: **SamplerPrecomp**

Algorithm 7 computes the message-independent data required by the signature sampler from a completed NTRU basis. To avoid floating-point arithmetic in signing while keeping efficiency, it computes ring-structure-preserving integer approximations of the inverse Gram–Schmidt basis and of the Gram root of the residual perturbation covariance. It stores the residual covariance Σ_δ , whose small coefficients are encoded as signed 22-bit fixed-point values, instead of materializing its dense inverse-covariance correction matrix. The goal is to use Σ_δ to rectify the Gaussian discrepancy caused by approximation errors during the signature. The algorithm returns

$$(\hat{u}, \hat{\mathbf{B}}^{-1}, \hat{\mathbf{A}}, \Sigma_\delta, r, C),$$

which is stored in the secret key and reused for all subsequent signing procedures. Note that Σ_δ can be computed using NTT with small modulus, say $< 2^{20}$, as its numerator is smaller than 16-bits.

One can choose to balance the trade-off between key generation time, secret key size, and signature time by deferring part, or all, of these precomputations to the signature.

Algorithm 7 | SamplerPrecomp

alg:sampler-precomp

Input

An NTRU basis $\mathbf{B} = [\mathbf{b}_1 \mid \mathbf{b}_2]$, a Gaussian parameter σ , a parameter η , and precisions p_1, p_2 .

Output

The precomputed signing data $(\hat{u}, \hat{\mathbf{B}}^{-1}, \hat{\mathbf{A}}, \Sigma_\delta, r, C)$.

- $u \leftarrow -\frac{\langle \mathbf{b}_1, \mathbf{b}_2 \rangle_K}{\langle \mathbf{b}_1, \mathbf{b}_1 \rangle_K}$
- $\hat{u} \leftarrow [u]_{p_1}$
- $\mathbf{U} \leftarrow [(1, 0), (\hat{u}, 1)]$
- $\hat{\mathbf{B}} \leftarrow \mathbf{B} \cdot \mathbf{U}^{-1}$
- $\hat{\mathbf{B}}^{-1} \leftarrow \mathbf{U} \cdot \mathbf{B}^{-1}$
- $\Sigma_p \leftarrow \sigma^2 \mathbf{I} - \hat{\mathbf{B}} \hat{\mathbf{B}}^t$
- $\mathbf{A} \leftarrow \sqrt{\Sigma_p - \mathbf{I}}$
- $\hat{\mathbf{A}} \leftarrow [\mathbf{A}]_{p_2}$
- $\Sigma_\delta \leftarrow \Sigma_p - \mathbf{I} - \hat{\mathbf{A}} \hat{\mathbf{A}}^t$
- $r \leftarrow \exp(-2\pi^2 \eta^2)$
- $C \leftarrow \exp\left(\frac{4d^2}{p_2 \cdot \sigma}\right)$
- return** $(\hat{u}, \hat{\mathbf{B}}^{-1}, \hat{\mathbf{A}}, \Sigma_\delta, r, C)$

$\triangleright$ ring-structure-preserving Cholesky decomposition

4.2 Design of **Sign**

4.2.1 Overview

Recall that an NTRU lattice is embedded in $\mathbb{R}^{2d}$. With the basis convention used in this specification, it is the kernel of the public map $\phi_h(x_1, x_2) = x_2 - hx_1 \bmod q$. Given a message M and a salt ω , the signer computes $\mathbf{m} = H(\omega \parallel M \parallel \text{pk}) \in R_q$, identifies $\mathbf{m}$ with its centered representative, and sets $\mathbf{c} = (0, \mathbf{m})$. Thus, $\phi_h(\mathbf{c}) = \mathbf{m}$.

The sampler returns a lattice point $\mathbf{v} = (\mathbf{v}_1, \mathbf{v}_2) \in \mathcal{L}_{\text{NTRU}}$ close to $\mathbf{c}$, and the signature vector is $(\mathbf{s}_1, \mathbf{s}_2) = \mathbf{c} - \mathbf{v}$. Since $\phi_h(\mathbf{v}) = 0$, the signature satisfies $\mathbf{s}_2 - h\mathbf{s}_1 = \mathbf{m} \bmod q$, or equivalently $\mathbf{s}_2 = \mathbf{m} + h\mathbf{s}_1 \bmod q$. The verifier can therefore reconstruct $\mathbf{s}_2$ from $\mathbf{s}_1$, $\mathbf{m}$, and h , so only $\mathbf{s}_1$ needs to be transmitted.

To obtain independently randomized hash targets across signing queries, **Sign** samples a salt $\omega \in \{0, 1\}^{\lambda+80}$ and hashes $\omega \parallel M \parallel \text{pk}$ rather than the message alone. As in the Falcon-style analysis, we set $\lambda + 80 = \lambda + \log_2(q_s)$, where λ is the target bit-security and $q_s = 2^{80}$ is the prescribed upper bound on the number of signing queries.

The parameter η is chosen as an upper bound on the smoothing parameter of $\mathbb{Z}$ under the weak-smoothness setting of [ZJR⁺25]. For the parameter sets of degrees $d = 512, 1024$, and 2048 , we use $\epsilon = 2^{-29}$, 2^{-26} , and $2^{-23.5}$, respectively. Accordingly, the smoothing standard deviation and the target signature standard deviation are set to

$$\eta = \frac{1}{\pi} \sqrt{\frac{1}{2} \log \left(2 \left(1 + \frac{1}{\epsilon} \right) \right)}, \quad \sigma_{\text{sig}} = \eta \cdot \sigma = \eta \cdot \chi \cdot \mathcal{Q}(f, g) \cdot \sqrt{q}. \quad (2)$$

Here, $\sigma = \chi \cdot \mathcal{Q}(f, g) \cdot \sqrt{q}$ is determined by the quality of the secret basis, and the scaling factor χ accounts for the quantization of the Gram–Schmidt coefficient during key generation. The parameter σ_{sig} determines both the typical signature length and the concrete hardness of forgery.

Finally, the norm-rejection threshold is chosen as

$$\gamma = \tau \cdot \sigma_{\text{sig}} \cdot \sqrt{2d}. \quad (3)$$

The concrete value of τ is given in Table 3. Thus, **Sign** retains only candidates satisfying $\|(\mathbf{s}_1, \mathbf{s}_2)\| \leq \gamma$.

Because $\mathbf{s}_1$ is a Gaussian vector, it admits a more compact representation than the raw coefficient encoding. At the selected Gaussian width, approximately $\lceil \log_2 \sqrt{q} \rceil$ low bits of each coefficient are close to

uniform and are therefore stored directly. Following [ETWY22, ZJR⁺25], we then encode all high parts jointly using rANS, which yields an additional compression gain while retaining efficient decoding. The resulting signature is of the form $\text{sig} = (\omega, \text{Compress}(\mathbf{s}_1))$.

The signing algorithm uses two rejection loops. The inner loop repeats preimage sampling until the complete signature vector satisfies the norm bound. The outer loop handles the variable length of the entropy encoding: if **Compress** returns $\perp$, signing restarts with a fresh salt and a new sample.

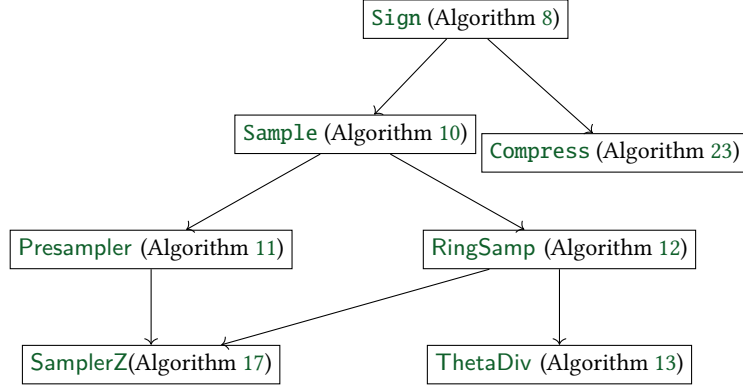


Fig. 3: Flowchart of **Sign**.

Algorithm 8 Sign		alg:sign
Input	A message $M \in \{0, 1\}^*$, a secret key sk , and a public key pk .	
Output	A signature (ω, str) with $\omega \in \{0, 1\}^{\lambda+80}$.	
1.	repeat	
2.	repeat	
3.	$\omega \leftarrow \mathcal{U}(\{0, 1\}^{\lambda+80})$	
4.	$\mathbf{m} \leftarrow \text{H}(\omega \parallel M \parallel \text{pk})$	
5.	$\mathbf{c} \leftarrow (0, \mathbf{m})$	
6.	$\mathbf{v} \leftarrow \text{Sample}(\mathbf{c}, \text{sk})$	$\triangleright \mathbf{v} \sim D_{\mathcal{L}_{\text{NTRU}}, \mathbf{c}, \sigma_{\text{sig}}}$
7.	$(\mathbf{s}_1, \mathbf{s}_2) \leftarrow \mathbf{c} - \mathbf{v}$	
8.	until $\ (\mathbf{s}_1, \mathbf{s}_2)\ ^2 \leq \gamma^2$	
9.	$\text{str} \leftarrow \text{Compress}(\mathbf{s}_1)$	
10.	until $\text{str} \neq \perp$	
11.	return (ω, str)	

$\triangleright \mathbf{v} \sim D_{\mathcal{L}_{\text{NTRU}}, \mathbf{c}, \sigma_{\text{sig}}}$

The four main subroutines correspond to the following stages of **Sign**.

1. The **Sample** algorithm implements an integer-arithmetic-friendly hybrid sampler. It combines perturbation sampling with two ring-level Gaussian decoding steps and applies two rejection corrections so that its output is distributed negligibly close to the target lattice Gaussian.
2. The **Presampler** algorithm samples the perturbation vector using the rounded Gram root $\hat{\mathbf{A}}$. It realizes the required covariance through the convolution of two coefficient-wise discrete Gaussian vectors.
3. The **RingSamp** algorithm performs discrete Gaussian sampling over R coefficient-wise. It also computes the center-dependent Gaussian-mass ratio required by the first rejection step, using **ThetaDiv**.
4. The **Compress** algorithm encodes $\mathbf{s}_1$ into a fixed-length string. If the representation exceeds the prescribed length, it returns $\perp$ and **Sign** restarts.

4.2.2 FPA-free hybrid preimage sampling: **Sample**

The hybrid sampler of [Pre15, EFG⁺22] combines these two approaches. Nearest-plane decoding is performed over the ring module, while each randomized rounding step is implemented by a Peikert-style ring sampler. Since an NTRU lattice has rank two as an R -module, only two ring-level decoding steps are required, instead of the $2d$ scalar steps appearing in an FFO sampler. The resulting procedure is simpler, faster, more parallelizable, and more amenable to protected implementations. The trapdoor-quality penalty traditionally associated with the hybrid sampler is largely mitigated by the high-quality bases generated using the Antrag technique [ENS⁺23].

Our implementation further avoids floating-point arithmetic during online sampling. The exact square Gram root of the perturbation covariance is computed during key generation and rounded to a fixed precision, yielding $\hat{\mathbf{A}}$. A ring-structure-preserving Cholesky decomposition ensures that multiplication by $\hat{\mathbf{A}}$ can still be implemented in quasi-linear time. The covariance discrepancy introduced by this quantization is then corrected by rejection sampling.

Algorithm 10 first invokes **Presampler** to generate a perturbation $\mathbf{p}$, transforms the residual target using $\hat{\mathbf{B}}^{-1}$, and applies **RingSamp** successively to the second and first ring coordinates. Writing $c'_1 = \hat{c}_1 - z_2 \hat{u}$ and $\mathbf{c}' = (\hat{c}_2, c'_1)$, let $\Sigma'_p = \hat{\mathbf{A}}\hat{\mathbf{A}}^t + \mathbf{I} = \Sigma_p - \Sigma_\delta$ and $\mathbf{T} = \sum_{i=1}^k (\Sigma_\delta / \sigma^2)^i$. Here, the matrix $\mathbf{T}$ is used for rectifying the Gaussian discrepancy caused by approximation errors

The two acceptance probabilities are

$$\begin{aligned} \Delta_1 &= \frac{\rho_{\eta, \hat{c}_2}(R)}{\rho_\eta(R)} \cdot \frac{\rho_{\eta, c'_1}(R)}{\rho_\eta(R)} = \frac{\rho_{\eta, \mathbf{c}'}(R^2)}{\rho_\eta(R^2)}, \\ \Delta_2 &= \frac{1}{C} \exp\left(\frac{(\mathbf{z} - \mathbf{c})^t \mathbf{T}(\mathbf{z} - \mathbf{c})}{2\sigma_{\text{sig}}^2}\right). \end{aligned} \quad (4)$$

Here $\mathbf{t} = \mathbf{T}(\mathbf{z} - \mathbf{c})$ is the action of the inverse-covariance correction on $\mathbf{x} = \mathbf{z} - \mathbf{c}$; it is computed from Σ_δ by Algorithm 9 below.

The first rejection corrects the center-dependent normalization factors introduced by the two calls to **RingSamp**. Without this correction, the output probability would contain $\rho_{\eta, \mathbf{c}'}(R^2)^{-1}$ and would therefore depend on the intermediate centers. Multiplication by Δ_1 replaces this quantity by the constant $\rho_\eta(R^2)^{-1}$, which is independent of the sampled vector. This is the correction that permits the hybrid sampler to operate under the weak-smoothness condition.

The second rejection step corrects the covariance error introduced by using the rounded Gram root $\hat{\mathbf{A}}$ instead of the exact one. Before this correction, the output density is, up to normalization, proportional to $\rho_{\eta\sqrt{\sigma^2\mathbf{I}-\Sigma_\delta}}(\mathbf{z} - \mathbf{c})$, where Σ_δ measures the covariance loss caused by rounding. Let

$$\mathbf{T} = \sum_{i=1}^k \Sigma_\delta^i / \sigma^{2i}, \text{ for a truncation order } k. \quad (5)$$

Since

$$(\mathbf{I} - \mathbf{S})^{-1} = \mathbf{I} + \sum_{i=1}^{\infty} \mathbf{S}^i = \mathbf{I} + \mathbf{T} + \sum_{i=k+1}^{\infty} \mathbf{S}^i,$$

multiplying by the correction factor

$$\exp\left(\frac{(\mathbf{z} - \mathbf{c})^t \mathbf{T}(\mathbf{z} - \mathbf{c})}{2\sigma_{\text{sig}}^2}\right)$$

transforms the density into

$$\rho_{\eta\sqrt{\sigma^2\mathbf{I}-\Sigma_\delta}}(\mathbf{z} - \mathbf{c}) \cdot \exp\left(\frac{(\mathbf{z} - \mathbf{c})^t \mathbf{T}(\mathbf{z} - \mathbf{c})}{2\sigma_{\text{sig}}^2}\right) \propto \rho_{\sigma_{\text{sig}}}(\mathbf{z} - \mathbf{c}) \cdot \exp\left(-\frac{(\mathbf{z} - \mathbf{c})^t \left(\sum_{i=k+1}^{\infty} \mathbf{S}^i\right) (\mathbf{z} - \mathbf{c})}{2\sigma_{\text{sig}}^2}\right).$$

By the choice of the truncation order k , the residual multiplicative factor is negligibly close to 1 on all retained candidates. Thus the corrected distribution is negligibly close to the target Gaussian density $\rho_{\sigma_{\text{sig}}}(\mathbf{z} - \mathbf{c})$.

Accordingly, Algorithm 10 uses the rejection probability

$$\Delta_2 = \frac{1}{C} \exp\left(\frac{(\mathbf{z} - \mathbf{c})^t \mathbf{T}(\mathbf{z} - \mathbf{c})}{2\sigma_{\text{sig}}^2}\right).$$

The constant C is chosen, so that $\Delta_2 \leq 1$ for every candidate reaching this step. In summary, Δ_1 removes the center-dependent normalization factors arising from arbitrary-centered ring sampling, while Δ_2 compensates for the covariance error introduced by quantizing the perturbation Gram root.

Materializing $\mathbf{T}_k$ would require storing another matrix with high precision, and computing a multiplication. Neither is necessary because the rejection test uses only its action on the current vector. If $\mathbf{t} = \mathbf{T}_k \mathbf{x}$, with $\mathbf{x} = \mathbf{z} - \mathbf{c}$, then Equation (5) is equivalent to the linear system

$$(\sigma^2 - \Sigma_\delta) \mathbf{t} = \sigma^2 \Sigma_\delta \mathbf{x}. \quad (6)$$

Algorithm 9 solves them with a fixed number ℓ_J of simultaneous Jacobi updates.

Algorithm 9 JacobiAction		alg:jacobi-action
Input	A residual covariance Σ_δ , a vector $\mathbf{x} \in K_{\mathbb{R}}^2$, the parameter σ , and an iteration count ℓ_J .	
Output	An approximation $\mathbf{t}^{(\ell_J)}$ to $(\mathbf{I} - \Sigma_\delta / \sigma^2)^{-1} \mathbf{x}$.	
1.	$\mathbf{x} \leftarrow \sigma^2 \Sigma_\delta \mathbf{x}$	
2.	for $\ell = 0$ to $\ell_J - 1$ do	
3.	$B \leftarrow \ \mathbf{t}_j\ _\infty / p_J$	
4.	$\mathbf{t}_j \leftarrow B \cdot \lfloor (\mathbf{x} / \sigma^2) / B \rfloor$ $\triangleright$ Rounding error of each coordinate is about $\ \mathbf{t}_j\ _\infty / p_J$ for some parameter p_J	
5.	$\mathbf{x} \leftarrow \mathbf{x} - (\sigma^2 - \Sigma_\delta) \mathbf{t}_j$	
6.	return $\sum_{j=0}^{\ell_J-1} \mathbf{t}_j$	

This algorithm corresponds to the expansion of $(\mathbf{I} - \Sigma_\delta / \sigma^2)^{-1}$, with the advantage that errors introduced by rounding at one step are corrected at the next step.

We empirically estimated ρ_{max} the spectral norm of Σ_δ / σ^2 for each parameter set by evaluating 10,000 signatures with different keys. The largest observed ρ_{max} was $1.712 \cdot 10^{-7}$, $1.738 \cdot 10^{-7}$ and $1.887 \cdot 10^{-7}$ for degree 512, 1024 and 2048 respectively. We can therefore take a small $1/p_J > \rho_{max}$, for example $p_J = 2^{20}$; then each iteration has to multiply the matrix Σ_δ , which has a 15 bit numerator by a 20 bit vector. Since σ has low precision, JacobiAction uses only small integers. An implementation tailored to embedded processor can take a smaller p_J .

The implementation targets an absolute error below 2^{-50} in the Δ_2 acceptance probability, we therefore fixed $\ell_J = 3$ for every parameter set.

In summary, Δ_1 removes the center-dependent normalization factors arising from arbitrary-centered ring sampling, while Δ_2 compensates for the covariance error introduced by quantizing the perturbation Gram root.

Algorithm 10 | Sample

alg:sample

Input A target $\mathbf{c} \in K_{\mathbb{R}}^2$, secret sampling data $(\mathbf{B}, \hat{\mathbf{B}}^{-1}, \hat{u}, \hat{\mathbf{A}}, \Sigma_{\delta}, C, r)$, and parameters L, k_{Θ} and B_{samp} .

Output A vector $\mathbf{z} \in \mathcal{L}_{\text{NTRU}}$ following a distribution negligibly close to $D_{\mathcal{L}_{\text{NTRU}}, \mathbf{c}, \sigma_{\text{sig}}}$.

1. $\mathbf{p} \leftarrow \text{Presampler}(\hat{\mathbf{A}}, \eta, L)$
2. $\hat{\mathbf{c}} \leftarrow \hat{\mathbf{B}}^{-1}(\mathbf{c} - \mathbf{p})$ $\triangleright \hat{\mathbf{c}} = (\hat{c}_1, \hat{c}_2)$
3. $(z_2, \Delta') \leftarrow \text{RingSamp}(\hat{c}_2, \eta, r, k_{\Theta})$
4. $c'_1 \leftarrow \hat{c}_1 - z_2 \hat{u}$
5. $(z_1, \Delta'') \leftarrow \text{RingSamp}(c'_1, \eta, r, k_{\Theta})$
6. $\mathbf{z} \leftarrow \mathbf{B}(z_1, z_2)$
7. $\Delta_1 \leftarrow \Delta' \Delta''$
8. $u_1 \leftarrow \mathcal{U}(0, 1)$
9. **if** $u_1 > \Delta_1$ **then** restart
10. $\mathbf{x} \leftarrow \mathbf{z} - \mathbf{c}$
11. $\mathbf{t} \leftarrow \text{JacobiAction}(\Sigma_{\delta}, \mathbf{x}, \sigma, 3)$
12. $\Delta_2 \leftarrow \frac{1}{C} \exp\left(\frac{\mathbf{x}^t \mathbf{t}}{2\sigma_{\text{sig}}^2}\right)$
13. $u_2 \leftarrow \mathcal{U}(0, 1)$
14. **if** $u_2 > \Delta_2$ **then** restart
15. **return** $\mathbf{z}$

4.2.3 Perturbation sampling: **Presampler**

Algorithm 11 samples the perturbation corresponding to the rounded covariance factor $\hat{\mathbf{A}}$. The scaling parameter L places the intermediate samples on the denser lattice $\frac{1}{L}R^2$, while the two Gaussian layers realize the covariance $\Sigma'_p = \hat{\mathbf{A}}\hat{\mathbf{A}}^t + \mathbf{I}$.

Algorithm 11 Presampler		alg:PreSampler
Input	A matrix $\hat{\mathbf{A}} \in \frac{1}{p}R^{2 \times 2}$, a standard deviation η , and an integer L .	
Output	A vector $\mathbf{p} \in \frac{1}{L}R^2$ distributed close to $D_{\frac{1}{L}R^2, \eta\sqrt{\Sigma'_p}}$, where $\Sigma'_p = \hat{\mathbf{A}}\hat{\mathbf{A}}^t + \mathbf{I}$.	
1. for $i = 1$ to $2d$ do 2. $x_i \leftarrow \text{SamplerZ}(0, L\eta)$ 3. $\mathbf{p}' \leftarrow \hat{\mathbf{A}}\mathbf{x}$ 4. for $i = 1$ to $2d$ do 5. $p_i \leftarrow \frac{1}{L}\text{SamplerZ}(Lp'_i, L\eta)$ 6. return $\mathbf{p}$		

4.2.4 Ring Gaussian sampling: **RingSamp**

Algorithm 12 samples a ring element coefficient-wise from one-dimensional discrete Gaussians. In addition to the sampled ring element, it returns the correction factor

$$\Delta = \frac{\rho_{\eta, \mathbf{c}}(R)}{\rho_{\eta}(R)} = \prod_{i=1}^d \frac{\rho_{\eta, c_i}(\mathbb{Z})}{\rho_{\eta}(\mathbb{Z})},$$

which is used in the first rejection step of **Sample**.

Algorithm 12 RingSamp		alg:RingSamp
Input	A center $\mathbf{c} = (c_1, \dots, c_d) \in R$, a standard deviation η , the precomputed value r , and a truncation order k_Θ .	
Output	A ring element $\mathbf{v} \in R$ and a correction factor $\Delta \approx \rho_{\eta, \mathbf{c}}(R) / \rho_\eta(R)$.	
<pre>1. $\Delta \leftarrow 1$ 2. for $i = 1$ to d do 3. $v_i \leftarrow \text{SamplerZ}(c_i, \eta)$ 4. $\Delta_i \leftarrow \text{ThetaDiv}(c_i, r, k_\Theta)$ 5. $\Delta \leftarrow \Delta \cdot \Delta_i$ 6. return $(\mathbf{v}, \Delta)$</pre>		

Theta-function division. Let $\Delta(c) = \rho_{\eta, c}(\mathbb{Z}) / \rho_\eta(\mathbb{Z})$. Using Poisson summation and the Jacobi triple-product identity, it can be expressed as

$$\Delta(c) = \prod_{i=1}^{\infty} \left(1 + \frac{2 \cos(2\pi c) - 2}{r^{2i-1} + r^{1-2i} + 2} \right), \quad r = \exp(-2\pi^2 \eta^2).$$

The product converges rapidly, so it is truncated after k_Θ terms. Since r is fixed for a parameter set, the powers of r and the corresponding denominators can be precomputed; only $\cos(2\pi c)$ must be evaluated online.

Algorithm 13 ThetaDiv		alg:ThetaDiv
Input	A center $c \in \mathbb{R}$, the precomputed value $r = \exp(-2\pi^2\eta^2)$, and a truncation order k_Θ .	
Output	An approximation of $\Delta(c) = \rho_{\eta,c}(\mathbb{Z}) / \rho_\eta(\mathbb{Z})$.	
1.	$\Delta \leftarrow 1$	
2.	$a \leftarrow \cos(2\pi c)$	
3.	for $i = 1$ to k_Θ do	
4.	$\Delta \leftarrow \Delta \cdot \left(1 + \frac{2a-2}{r^{2i-1} + r^{1-2i} + 2} \right)$	
5.	return Δ	

4.3 Design of Verif:

The verification is much simpler. On receiving a signature (ω, s) and a message M , the verifier decompresses s into a polynomial s_1 , computes $c = H(\omega \| M \| \mathbf{pk})$, and reconstructs the full signature vector $\mathbf{s} = (s_1, s_2)$. The defining congruence is

$$s_2 - h s_1 \equiv c \pmod{q},$$

and hence the verifier computes

$$s_2 \equiv c + h s_1 \pmod{q}.$$

This computation takes place in R_q and, for a suitable choice of modulus q , can be carried out very efficiently via the Number Theoretic Transform (NTT). Finally, the verifier checks that $\|(\mathbf{s}_1, \mathbf{s}_2)\|^2 \leq \gamma^2$ and accepts the signature only in this case.

The rejection bound γ is derived from the expected length of the vectors produced by **Sample**. Since these are morally Gaussian, they concentrate around their standard deviation; the slack parameter τ is tuned so that the overwhelming majority of the vectors generated by **Sample** pass the rejection step: $\gamma = \tau \cdot \sigma_{\text{sig}} \cdot \sqrt{2d}$.

Algorithm 14 Verif	alg:verif
Input A signature (ω, s) on M , a public key $\text{pk} = (q, h, \sigma_{\text{sig}}, \eta, \gamma)$. Output Accept or reject.	
1. $s_1 \leftarrow \text{Decompress}(s)$ 2. if $s_1 = \perp$ then Reject 3. $c \leftarrow H(\omega \parallel M \parallel \text{pk})$ 4. $s_2 \leftarrow c + hs_1 \bmod q$ 5. if $\ (s_1, s_2)\ ^2 > \gamma^2$ then return Reject 6. return Accept	

4.4 Miscellaneous

This section gathers the auxiliary algorithms invoked by **KeyGen**, **Sign**, and **Verif**.

4.4.1 Specifications of **FFT** and **FFT⁻¹**:

Algorithms 15 and 16 describe, respectively, the FFT over $K_{\mathbb{R}}$ and its inverse.

Algorithm 15 FFT	alg:FFT
Input $f \in K_{\mathbb{R}} = \mathbb{R}[X]/(X^d + 1)$. Output The FFT representation of f .	
1. $\zeta = \exp(i\pi/d)$ 2. $\varphi(f) \leftarrow (f(\zeta^1), f(\zeta^3), \dots, f(\zeta^{2d-1}))$ 3. return $\varphi(f)$	

Algorithm 16 FFT ⁻¹	alg:FFTinv
Input $\mathbf{c} = (c_0, \dots, c_{d-1}) \in \mathbb{C}^d$ such that $c_{d-1-i} = \overline{c_i}$. Output $f \in K_{\mathbb{R}}$ such that $\mathbf{c} = \varphi(f)$.	
1. $\zeta = \exp(i\pi/d)$ 2. $\mathbf{V} = \left(\overline{\zeta^{jk}} \right)_{j \in \mathbb{Z}_d, k \in \mathbb{Z}_{2d}^*}$ 3. $f \leftarrow \frac{1}{d} \cdot \mathbf{V}\mathbf{c}$ 4. return f	

4.4.2 Integer Gaussian sampling: **SamplerZ**

The signing sampler requires arbitrary-centered integer Gaussian sampling at the two fixed standard deviations η and $L\eta$, where $L = 4$. To describe both cases uniformly, we write $\tilde{\eta} \in \{\eta, L\eta\}$. Our construction follows the integer Gaussian sampler of FALCON [PFH⁺22], specialized to the two fixed widths used by YUANYANG.DSA.

Algorithm 17 obtains an arbitrary-centered sample by combining two subroutines: the half-Gaussian sampler **BaseSampler** and the Bernoulli rejection test **BerExp**.

Algorithm 17 SamplerZ		alg:SamplerZ
Input	A center $\mu \in \mathbb{R}$ and a standard deviation $\bar{\eta} \in \{\eta, L\eta\}$.	
Output	An integer $z \in \mathbb{Z}$ sampled from a distribution close to $D_{\mathbb{Z}, \mu, \bar{\eta}}$.	
<pre>1. $a \leftarrow \lfloor \mu \rfloor$ 2. $c \leftarrow \mu - a$ 3. while 1 do 4. $z_0 \leftarrow \text{BaseSampler}(\bar{\eta})$ 5. $b \leftarrow \mathcal{U}(\{0, 1\})$ 6. $z \leftarrow b + (2b - 1)z_0$ 7. $x \leftarrow \frac{(z - c)^2 - z_0^2}{2\bar{\eta}^2}$ 8. if $\text{BerExp}(x, 1) = 1$ then return $z + a$</pre>		

RCDT base sampling. Algorithm 18 samples from the centered half-Gaussian $D_{\mathbb{Z}, \bar{\eta}}^+$ using a 96-bit reverse cumulative distribution table $\text{RCDT}_{\bar{\eta}}$. For each parameter set, separate tables are provided for $\bar{\eta} = \eta$ and $\bar{\eta} = 4\eta$.

Algorithm 18 BaseSampler		alg:BaseSampler
Input	A standard deviation $\bar{\eta} \in \{\eta, L\eta\}$.	
Output	A nonnegative integer z_0 sampled from a distribution close to $D_{\mathbb{Z}, \bar{\eta}}^+$.	
<pre>1. $u \leftarrow \mathcal{U}(\{0, \dots, 2^{96} - 1\})$ 2. $z_0 \leftarrow 0$ 3. for $i = 0$ to $\text{len}(\text{RCDT}_{\bar{\eta}}) - 1$ do 4. $z_0 \leftarrow z_0 + \llbracket u < \text{RCDT}_{\bar{\eta}}[i] \rrbracket$ 5. return z_0</pre>		

In the reference implementation, every 96-bit threshold is stored as three 32-bit limbs. Tables 1 and 2 list the corresponding complete integer thresholds for all parameter sets.

Table 1: RCDT thresholds used by **BaseSampler** at standard deviation η .

i	$\text{RCDT}_{\eta}[i]$		
	$d = 512$ $\eta = 1.026$	$d = 1024$ $\eta = 0.974$	$d = 2048$ $\eta = 0.899$
0	34865012908650868144707915501	33184765138833518093412043706	31588619392639700572488302970
1	7275680111900916072112508013	6003334535413659071708200966	4931210010448158513954395550
2	639813060982194083955722052	411077055761821968350987348	260649502342656826968743039
3	22519414226109127103122033	10107031860821288740123852	4424933144200052363665142
4	310691074910044736286254	87520252750386145193961	23711420554445033494971
5	1666494246248217449628	265097006637069927572	39884972986093141857
6	3464131885340719787	280201182929081019	21023577635896876
7	2787163525452926	103261853947769	3470664864187
8	867560125521	13264403122	179412737
9	104454032	593839	2904
10	4864	9	–

Table 2: RCDT thresholds used by [BaseSampler](#) at standard deviation 4η .

i	RCDT $_{4\eta}[i]$		
	$d = 512$ $4\eta = 4.104$	$d = 1024$ $4\eta = 3.896$	$d = 2048$ $4\eta = 3.596$
0	65189579802234483245806349635	64509704494613519003435694854	63850926995224426193104732411
1	51561624774198811465316619118	50268183552486540306059106821	49021686337848591519756024799
2	39094882779319188737911676172	37366732742758300980661039798	35722016661244523262505610059
3	28347812106349510342704182308	26424426944894272440191476808	24629142403956998198386088286
4	19617252308857507705590438139	17735477648030101925850863040	16024583343621402856826192417
5	12933675691266905580348321346	11275756110523620537400095462	9817401852675083142425124655
6	8112081847357840100943486700	6779526826994225867185622697	5653106137683492812801537935
7	4834246228491509074986080991	3849502533121790597675416392	3054921715989399003263518718
8	2734346440299545797520364445	2061854104874944651978079096	1547344098049058224006187599
9	1466622118438790972514532555	1040723905965223490022891997	733819513069945270657907720
10	745405162335652034327687446	494628846210489013668729045	325553735607354821864418690
11	358751713824355446596047792	221200526322343481656259045	135009170300551134927762198
12	163411031419351021426348632	93024483711846538022165491	52304230493083819972714065
13	70412012631826631836512227	36769844677110298494241167	18919518985797699072817360
14	28688690052092793535099065	13654629288802444698297338	6386805835760000870234399
15	11048865632958460386323264	4762092603806501906791752	2011344345995864380119476
16	4020983265309121445603064	1559206427134231966249850	590704713847661058129119
17	1382407540015860454855155	479152442753771049033867	161736333805566255124117
18	448874926532056705794701	138165663876854536411712	41274927308148870083877
19	137628567053027894985806	37375596383132908110694	9815423193275698251308
20	39838685715339722601810	9483185744083096486965	2174653490591487827141
21	10885362990803674034764	2256444858315240890052	448801127680964042958
22	2807104300414213809913	503424195606037096723	86264853456435708375
23	683116807559170331698	105298961076191368018	15440831289465535648
24	156856025341724616114	20646323431023682939	2573425378205109846
25	33980634559817011091	3794402903101019785	399309390656089231
26	6944557823802695194	653558451086170971	57679617750363150
27	1338761917492445003	105494075133848526	7755544804570260
28	243429989889564019	15956601191152904	970611231555775
29	41747095565911621	2261469262337600	113055240460699
30	6751995741732289	300297093655128	12255209653836
31	1029832259716469	37359102048791	1236261527981
32	148117441963891	4354151992435	116047759010
33	20087699846889	475391734482	10136295669
34	2568734431832	48620722141	823794830
35	309710181244	4657956281	62292958
36	35206492902	417982013	4382511
37	3773167493	35131219	286849
38	381234017	2765594	17465
39	36313382	203904	988
40	3260763	14078	51
41	276017	909	2
42	22023	54	–
43	1655	2	–
44	116	–	–
45	7	–	–

Fixed-point exponential approximation. Algorithm 19 implements the fixed-point polynomial approximation of $\text{ccs} \cdot \exp(-x)$ required by [BerExp](#). The approximation coefficients are inherited from the Falcon sampler.

Algorithm 19 ApproxExp		alg:ApproxExp
Input Values $x \in [0, \ln 2]$ and $ccs \in [0, 1]$. Output An integer approximation of $2^{63} \cdot ccs \cdot \exp(-x)$.		
<pre> 1. $C \leftarrow \begin{bmatrix} 0x00000004741183A3, 0x00000036548CFC06, 0x0000024FDCBF140A, \\ 0x0000171D939DE045, 0x0000D00CF58F6F84, 0x000680681CF796E3, \\ 0x002D82D8305B0FEA, 0x011111110E066FD0, 0x0555555555070F00, \\ 0x155555555581FF00, 0x400000000002B400, 0x7FFFFFFFFFFFF4800, \\ 0x8000000000000000 \end{bmatrix}$ 2. $y \leftarrow C[0]$ 3. $z \leftarrow \lfloor 2^{63} \cdot x \rfloor$ 4. for $u = 1$ to 12 do 5. $y \leftarrow C[u] - ((z \cdot y) \gg 63)$ 6. $z \leftarrow \lfloor 2^{63} \cdot ccs \rfloor$ 7. $y \leftarrow (z \cdot y) \gg 63$ 8. return y </pre>		

Bernoulli exponential sampling. Algorithm 20 returns 1 with probability approximately $ccs \cdot \exp(-x)$ using **ApproxExp** and a byte-wise comparison with uniform random bits. In Algorithm 17, it is invoked with $ccs = 1$.

Algorithm 20 BerExp		alg:BerExp
Input Values $x \geq 0$ and $ccs \in [0, 1]$. Output A bit equal to 1 with probability approximately $ccs \exp(-x)$.		
<pre> 1. $s \leftarrow \lfloor x / \ln 2 \rfloor$ 2. $r \leftarrow x - s \cdot \ln 2$ 3. $s \leftarrow \min(s, 63)$ 4. $z \leftarrow (2 \cdot \text{ApproxExp}(r, ccs) - 1) \gg s$ $\triangleright z \approx 2^{64} ccs \exp(-x)$ 5. $i \leftarrow 64$ 6. do 7. $i \leftarrow i - 8$ 8. $w \leftarrow \text{UniformBits}(8) - ((z \gg i) \& 0xFF)$ 9. while $(w = 0) \wedge (i > 0)$ 10. return $\llbracket w < 0 \rrbracket$ </pre>		

4.4.3 Specifications of **Compress** and **Decompress**:

We use an rANS-based encoding following the strategy of [ZJR⁺25]. Each coefficient is decomposed into a sign bit, ℓ low bits, and a high part. The sign bits and low bits are stored directly, while the high parts are compressed by rANS. The parameter ℓ controls the compression level.

We also adopt the secure rANS checks of [ZJR⁺25]: the encoder starts from a fixed public state, and the decoder accepts only if the final state is correct and the rANS input is fully consumed. The output is padded to the fixed length $slen$, so no extra length field is needed.

Algorithm 21 | rANSEncode

alg:ransencode

Input The radix b , the state interval I , the fixed initial state $x_{\text{init}} \in I$, the packed-state width w , the symbol intervals $\{I_a\}_{a \in \mathcal{H}}$, and symbols $(a_1, \dots, a_n) \in \mathcal{H}^n$.

Output A rANS string, or $\perp$.

```
1.  $x \leftarrow x_{\text{init}}$ 
2.  $j \leftarrow -1$ 
3. for  $i = n$  downto 1 do
4.   if  $a_i \notin \mathcal{H}$  then return  $\perp$ 
5.   while  $x \notin I_{a_i}$  do
6.      $str_j \leftarrow x \bmod b$ 
7.      $j \leftarrow j - 1$ 
8.      $x \leftarrow \lfloor x/b \rfloor$ 
9.    $x \leftarrow C(a_i, x)$ 
10.   $(str_{j-w+1}, \dots, str_j) \leftarrow \text{PackState}_w(x)$ 
11. return  $(str_{j-w+1}, \dots, str_j, str_{j+1}, \dots, str_{-1})$ 
```

Algorithm 22 rANSDecode		alg:ransdecode
Input	The radix b , the state interval I , the fixed initial state $x_{\text{init}} \in I$, the packed-state width w , the number of symbols n , and a rANS string $(str_1, \dots, str_N)$.	
Output	The decoded symbols $(a_1, \dots, a_n) \in \mathcal{H}^n$, or $\perp$.	
<pre>1. if $w > N$ then return $\perp$ 2. $x \leftarrow \text{UnpackState}_w(str_1, \dots, str_w)$ 3. $j \leftarrow w + 1$ 4. if $x \notin I$ then return $\perp$ 5. for $i = 1$ to n do $(a_i, x) \leftarrow D(x)$ 6. if $a_i \notin \mathcal{H}$ then return $\perp$ 7. while $x \notin I$ do 8. if $j > N$ then return $\perp$ 9. $x \leftarrow b \cdot x + str_j$ 10. $j \leftarrow j + 1$ 11. if $x \neq x_{\text{init}}$ then return $\perp$ 12. if $j \neq N + 1$ then return $\perp$ 13. return $(a_1, \dots, a_n)$</pre>		

Algorithm 23 Compress	alg:compress
Input A polynomial $s = \sum_{i=0}^{d-1} s_i X^i \in R = \mathbb{Z}[X]/(X^d + 1)$, an integer ℓ, and an integer $slen$. Output A compressed representation str of s of bitsize $slen$, or $\perp$.	
<pre> 1. $\vec{h} \leftarrow ()$ 2. $str_{aux} \leftarrow \{\}$ 3. for $i = 0$ to $d - 1$ do 4. $a_i \leftarrow s_i$ 5. $\sigma_i \leftarrow 1$ if $s_i < 0$, and $\sigma_i \leftarrow 0$ otherwise 6. $l_i \leftarrow a_i \bmod 2^\ell$ 7. $h_i \leftarrow a_i \gg \ell$ 8. if $h_i \notin \mathcal{H}$ then return $\perp$ 9. $\vec{h} \leftarrow \vec{h} \parallel h_i$ 10. $str_{aux} \leftarrow str_{aux} \parallel \sigma_i \parallel \text{bin}_\ell(l_i)$ 11. $str_{rANS} \leftarrow \text{rANSEncode}_{rp}(\vec{h})$ 12. if $str_{rANS} = \perp$ then return $\perp$ 13. $str \leftarrow str_{rANS} \parallel str_{aux} \parallel 1$ 14. if $str > slen$ then return $\perp$ 15. $str \leftarrow str \parallel 0^{slen - str }$ 16. return str </pre>	

Algorithm 24 Decompress	alg:decompress
Input A bitstring str of bitsize $slen$ and an integer ℓ. Output A polynomial $s = \sum_{i=0}^{d-1} s_i X^i \in R = \mathbb{Z}[X]/(X^d + 1)$, or $\perp$.	
<pre> 1. if $str \neq slen$ then return $\perp$ 2. Parse str as $str' \parallel 1 \parallel 0^t$ for some $t \geq 0$, where the displayed 1 is the last nonzero bit of str 3. if no such bit exists then return $\perp$ 4. if $str' < d(\ell + 1)$ then return $\perp$ 5. $str_{rANS} \leftarrow str'[0 : str' - d(\ell + 1)]$ 6. $str_{aux} \leftarrow str'[str' - d(\ell + 1) :]$ 7. $(h_0, \dots, h_{d-1}) \leftarrow \text{rANSDecode}_{rp}(str_{rANS}, d)$ 8. if $(h_0, \dots, h_{d-1}) = \perp$ then return $\perp$ 9. for $i = 0$ to $d - 1$ do 10. Parse the next $\ell + 1$ bits of str_{aux} as $\sigma_i \parallel \text{bin}_\ell(l_i)$ 11. $a_i \leftarrow (h_i \ll \ell) + l_i$ 12. if $a_i = 0$ and $\sigma_i = 1$ then return $\perp$ 13. $s_i \leftarrow -a_i$ if $\sigma_i = 1$, and $s_i \leftarrow a_i$ otherwise 14. return $s = \sum_{i=0}^{d-1} s_i X^i$ </pre>	

4.5 List of parameters

For convenience, the concrete values of all parameters introduced in this section are summarised in Table 3.

Table 3: List of parameters for YUANYANG.DSA

	YUANYANG.DSA-512	YUANYANG.DSA-1024	YUANYANG.DSA-2048
ring degree d	512	1024	2048
dimension $2d$	1024	2048	4096
modulus q	2689	4481	7681
salt length $\lambda + 80$	208	336	592
smoothing η	1.026	0.974	0.928
smoothness ϵ	2^{-29}	2^{-26}	$2^{-23.5}$
scaling χ	1.016	1.012	1.017
precision (p_1, p_2)	$(2^{11}, 2^{22})$	$(2^{11}, 2^{22})$	$(2^{11}, 2^{22})$
repetitions M	1.1	1.1	1.1
Norm _{fg}	2840	5076	9000
LossBound	3.8	22	128
quality α	1.29	1.4	1.43
lower radius R_-	49.1	63.12	82.63
upper radius R_+	58.0	78.41	103.98
signature width σ_{sig}	69.768	92.40825	128.262
σ	68	1518/16	255/2
slack τ	1.047	1.033	1.010
rejection bound γ	2337	4317	7641

5 Security Statements and Analyses

5.1 Provable security

The provable security of YUANYANG.DSA relies on the following hardness assumption.

Definition 5.1 (t - $\mathcal{R}$ -ISIS). *The Ring Inhomogeneous Short Integer Solution problem relative to the NTRU trapdoor algorithm TpdGen with parameters $m, q > 0$ and $\alpha, \beta > 0$ is defined via the game t - $\mathcal{R}$ -ISIS in Fig. 4. The advantage of A is defined as*

$$\text{Adv}_{m,q,\alpha,\beta,A}^{t\text{-}\mathcal{R}\text{-ISIS}} = \Pr[t\text{-}\mathcal{R}\text{-ISIS}_{m,q,\alpha,\beta}(A) \Rightarrow 1].$$

Game $t\text{-}\mathcal{R}\text{-ISIS}_{m,q,\alpha,\beta}(A)$	
1:	for $i \in [m]$ do
2:	$(h_i, \cdot, \cdot) \leftarrow \text{TpdGen}(q, \alpha)$
3:	for $j \in [t]$ do
4:	$c_j \leftarrow \mathcal{R}_q$
5:	$(j, u, v_1, \dots, v_m) \leftarrow A(h_1, \dots, h_m, c_1, \dots, c_t)$
6:	return $\llbracket u - \sum_i h_i \cdot v_i = c_j \wedge \ (u, v_1, \dots, v_m)\ \leq \beta \rrbracket$

Fig. 4: Games defining $t\text{-}\mathcal{R}\text{-ISIS}_{m,q,\alpha,\beta}$

Theorem 5.2 (Unforgeability). *Let A be any adversary against the **UF-CMA** security of YUANYANG.DSA running in time t_A , making at most Q_s signing queries and Q_H random oracle queries. Then there exists an adversary B against $(Q_H + 1)\text{-}\mathcal{R}\text{-ISIS}$ running in time $t_B \approx t_A$ such that, for all $C_s \in \mathbb{N}^{\geq 1}$ and $a_p, a_u \in \mathbb{R}^{>1}$,*

$$\text{Adv}_{\text{YUANYANG.DSA}, \mathbf{A}}^{Q_s\text{-UF-CMA}} \leq \left(r_u^{C_s} \cdot \left(r_p^{C_s} \cdot \text{Adv}_{1,q,\alpha,\beta,\mathbf{B}}^{(Q_H+1)\text{-}\mathcal{R}\text{-ISIS}} \right)^{\frac{ap-1}{ap}} \right)^{\frac{au-1}{au}} + \sum_{i=0}^{Q_s-1} \binom{C_s}{i} (1 - p_{\text{Sample},\beta})^{C_s-i} (p_{\text{Sample},\beta})^i + \frac{Q_s(C_s + Q_H)}{2^k},$$

where

- $p_{\text{Sample},\beta}$ is the minimum, over $(\mathbf{B}, \cdot) \in \text{TpGen}$ and $c \in \mathcal{R}_q$, of

$$\Pr_{(s_1, s_2) \leftarrow \text{Sample}(\mathbf{B}, s, (0, c))} [\|(s_1, s_2)\| \leq \beta].$$

- $r_u = R_{a_u}(\mathcal{P} \parallel \mathcal{Q}_h)$ with $\mathcal{P} = \mathcal{U}(\mathcal{R}_q)$ and $\mathcal{Q}_h$ the distribution of $s_2 - h s_1 \pmod q$, where $s_1, s_2 \sim D_{\mathcal{R}, s}$ and $(\cdot, h) \in \text{TpGen}$
- $r_p = R_{a_p}(\text{Sample}(\mathbf{B}, s, (0, c)) \parallel D_{\Lambda(\mathbf{B}), s, (0, c)})$.

Proof. Consider the sequence of games defined in Fig. 5.

Game G_0 . This is the unforgeability game for YUANYANG.DSA. We have

$$\Pr[G_0^A \Rightarrow 1] = \text{Adv}_{\text{YUANYANG.DSA}, \mathbf{A}}^{Q_s\text{-UF-CMA}}.$$

Game G_1 . This game is identical to the previous one, except that it aborts whenever the total number of preimages sampled by the signing oracle — counting repetitions — exceeds the threshold C_s .

Claim. For $p_{\text{Sample},\beta} = \min_{c \in \mathcal{R}_q} \Pr_{(s_1, s_2) \leftarrow \text{Sample}(\mathbf{B}, s, (0, c))} [\|(s_1, s_2)\|_2 \leq \beta]$ it holds that

$$\left| \Pr[G_0^A \Rightarrow 1] - \Pr[G_1^A \Rightarrow 1] \right| \leq \sum_{i=0}^{Q_s-1} \binom{C_s}{i} (1 - p_{\text{Sample},\beta})^{C_s-i} (p_{\text{Sample},\beta})^i.$$

Proof. Model the experiment by a binomial random variable $X \sim B(C_s, p_{\text{Sample},\beta})$: we run C_s Bernoulli trials of success probability $p_{\text{Sample},\beta}$, where each trial corresponds to a preimage produced by Sample in the signing oracle, and is declared successful when $\|(s_1, s_2)\|_2 \leq \beta$. The total number of successes equals the number of signing queries we can answer; since we must answer Q_s queries, the bound of interest is $\Pr[X < Q_s]$, which is exactly the claim. $\square$

Game G_2 . This game is identical to the previous one, except that the signing oracle Sign aborts whenever the random oracle has already been queried on the same public key, salt r , and message M . To simplify subsequent hybrids, we introduce a new random oracle H' that shares the table $\mathcal{H}$ with H but aborts on inputs already seen; the signing oracle then calls H' in place of H .

Claim. $|\Pr[G_1^A \Rightarrow 1] - \Pr[G_2^A \Rightarrow 1]| \leq \frac{Q_s(C_s + Q_H)}{2^k}$.

Proof. The salt r is drawn uniformly at random from $\{0, 1\}^k$ for every signing query. The set $\mathcal{H}$ contains at most $C_s + Q_H$ entries, since each query to H or H' adds at most one entry, and there are at most C_s implicit queries via Sign together with Q_H direct ones. Hence the probability that the freshly chosen salt collides with a previous query is at most $(C_s + Q_H)/2^k$, and summing over Q_s queries to Sign yields the claimed bound. $\square$

Game G_3 . This game is identical to the previous one, except that the random oracle H' no longer returns a uniformly random element $c \leftarrow \mathcal{R}_q$. Instead, it samples $s_1, s_2 \leftarrow D_{\mathcal{R}, s}$ and sets $c = s_2 - h \cdot s_1 \pmod q$, where h is the public key. For later use, s_1 and s_2 are returned alongside the random-oracle output (recall that H' is not exposed to the adversary).

Claim. For $h \in \mathcal{R}$, let $\mathcal{P} = \mathcal{U}(\mathcal{R}_q)$ and let $\mathcal{Q}_h$ be the distribution of $s_2 - h \cdot s_1 \pmod q$, where $s_1, s_2 \leftarrow D_{\mathcal{R}, s}$. Then, for any $a_u \in (1, \infty)$,

$$\Pr[G_2^A \Rightarrow 1] \leq \left(R_{a_u}(\mathcal{P} \parallel \mathcal{Q}_h)^{C_s} \cdot \Pr[G_3^A \Rightarrow 1] \right)^{\frac{au-1}{au}}$$

Fig. 5: Games for the proof of Theorem 5.2.

Games $G_0 - G_5$ 1: $\mathcal{H}, \mathcal{Q} \leftarrow \emptyset$ 2: $cnt \leftarrow 0$ 3: $(\mathbf{B}, h) \leftarrow \text{Gen}$ 4: $(M^*, \text{sig}^*) \leftarrow \text{ASign}(\cdot, \text{H}(\cdot, \cdot, \cdot))(h)$ 5: return $\llbracket \text{Verify}(h, M^*, \text{sig}^*) = 1 \wedge (M^*, \cdot) \notin \mathcal{Q} \rrbracket$	
Oracle $\text{Sign}(M)$ 1: repeat 2: $cnt \leftarrow cnt + 1$ / $G_1 - G_5$ 3: if $cnt > C_s$ / $G_1 - G_5$ 4: abort / $G_1 - G_5$ 5: $r \leftarrow \{0, 1\}^k$ 6: $c = \text{H}(h, r, m)$ / $G_0 - G_1$ 7: $(c, s_1, s_2) \leftarrow \text{H}'(h, r, m)$ / $G_2 - G_5$ 8: $(s_1, s_2) \leftarrow \text{Sample}(\mathbf{B}, s, (0, c))$ / $G_0 - G_3$ 9: $(s_1, s_2) \leftarrow D_{\Lambda(\mathbf{B}), s, (0, c)}$ / G_4 10: until $\ (s_1, s_2)\ \leq \beta$ 11: $\text{sig} \leftarrow (r, s_1)$ 12: $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(M, \text{sig})\}$ 13: return sig	
Oracle $\text{H}'(h, r, M)$ 1: if $\exists c \text{ s.t. } (c, h, r, M) \in \mathcal{H}$ 2: abort 3: $c \leftarrow \mathcal{R}_q$ 4: $(s_1, s_2) \leftarrow (\perp, \perp)$ 5: $s_1, s_2 \leftarrow D_{\mathcal{R}, s}$ / $G_3 - G_5$ 6: $c \leftarrow s_2 - h \cdot s_1 \bmod q$ / $G_3 - G_5$ 7: $\mathcal{H} \leftarrow \mathcal{H} \cup \{(c, h, r, M)\}$ 8: return (c, s_1, s_2)	Oracle $\text{H}(h, r, M)$ 1: if $\exists c \text{ s.t. } (c, h, r, M) \in \mathcal{H}$ 2: return c 3: $c \leftarrow \mathcal{R}_q$ 4: $\mathcal{H} \leftarrow \mathcal{H} \cup \{(c, h, r, M)\}$ 5: return c

Proof. Define two underlying distributions for a $(Q + 1)$ -tuple of random variables $(c_0 = (\mathbf{B}, h), c_1, \dots, c_Q)$.

$$\begin{array}{ll}
\tilde{\mathcal{P}} & \tilde{\mathcal{Q}} \\
(\mathbf{B}, h) \xleftarrow{s} \text{TpGen} & (\mathbf{B}, h) \leftarrow \text{TpGen} \\
\text{for } i \in [Q] & \text{for } i \in [Q] \\
\quad c_i \leftarrow \mathcal{R}_q & \quad (s_1, s_2) \leftarrow D_{\mathcal{R}, s} \\
\text{return } ((\mathbf{B}, h), c_1, \dots, c_Q) & \quad c_i = s_2 - h \cdot s_1 \pmod{q} \\
& \text{return } ((\mathbf{B}, h), c_1, \dots, c_Q)
\end{array}$$

These distributions describe the underlying distributions of G_2 and G_3 . By the data processing inequality it holds that, for any $a \in (1, \infty)$,

$$R_a(G_2 \parallel G_3) \leq R_a(\tilde{\mathcal{P}} \parallel \tilde{\mathcal{Q}}).$$

Let the marginal distribution of c_i be denoted by $\tilde{\mathcal{P}}_i$ ($\tilde{\mathcal{Q}}_i$ resp.) and the distribution of c_i conditioned on $c_{<i} = (c_0, \dots, c_{i-1})$ as $\tilde{\mathcal{P}}_{i|c_{<i}}$ ($\tilde{\mathcal{Q}}_{i|c_{<i}}$ resp.). Since the distribution of $c_0 = (\mathbf{B}, h)$ is the same for $\tilde{\mathcal{P}}$ and $\tilde{\mathcal{Q}}$, it holds that

$$R_a(\tilde{\mathcal{P}}_0 \parallel \tilde{\mathcal{Q}}_0) = 1.$$

For the conditional distributions, note that random variable c_i is independent of the previous random variables $c_1, \dots, c_{i-1}$. However, c_i might depend on h and thus on random variable $c_0 = (\mathbf{B}, h)$. Hence for all $i \in [Q + 1]$,

$$\begin{aligned}
R_a(\tilde{\mathcal{P}}_{i|c_{<i}} \parallel \tilde{\mathcal{Q}}_{i|c_{<i}}) &= R_a(\tilde{\mathcal{P}}_{i|(c_0, \dots, c_{i-1})} \parallel \tilde{\mathcal{Q}}_{i|(c_0, \dots, c_{i-1})}) \\
&= R_a(\mathcal{P} \parallel \mathcal{Q}_{i|c_0}) \\
&= R_a(\mathcal{P} \parallel \mathcal{Q}_h),
\end{aligned}$$

where $\mathcal{P} = \mathcal{U}(\mathcal{R}_q)$ and $\mathcal{Q}_h$ is the distribution of $s_2 - h \cdot s_1 \pmod{q}$ where $s_1, s_2 \leftarrow D_{\mathcal{R}, s}$. Note that h does not occur in distribution $\mathcal{P}$ because the individual random variables c_i (for $i \geq 1$) are independent of h . It then follows that

$$R_a(\tilde{\mathcal{P}} \parallel \tilde{\mathcal{Q}}) \leq R_a(\mathcal{P} \parallel \mathcal{Q}_h)^Q.$$

Finally, we obtain

$$\Pr[G_3^A \Rightarrow 1] \geq \frac{\Pr[G_2^A \Rightarrow 1]^{\frac{a}{a-1}}}{R_a(G_2 \parallel G_3)} \geq \frac{\Pr[G_2^A \Rightarrow 1]^{\frac{a}{a-1}}}{R_a(\mathcal{P} \parallel \mathcal{Q}_h)^Q}.$$

The claim follows by setting $Q = C_s$ due to at most C_s queries from Sign to H' .

Game G_4 . This game is identical to the previous one, except that the output of the preimage sampler $\text{Sample}(\mathbf{B}, s, (0, c))$ is replaced by an exact sample from $D_{\Lambda(\mathbf{B}), s, (0, c)}$.

Claim. For distributions $\text{Sample} = \text{Sample}(\mathbf{B}, s, (0, c))$, $D = D_{\Lambda(\mathbf{B}), s, (0, c)}$, and $a_p \in (1, \infty)$ it holds that

$$\Pr[G_3^A \Rightarrow 1] \leq \left(R_{a_p}(\text{Sample} \parallel D)^{C_s} \cdot \Pr[G_4^A \Rightarrow 1] \right)^{\frac{ap-1}{ap}}.$$

Proof. The claim follows analogously to Game G_3 .

Game G_5 . This game is identical to the previous one, except that s_1, s_2 are no longer sampled from a Gaussian distribution centred at $(0, c)$. Instead, the signing oracle reuses the preimages of c already sampled inside H or H' .

Claim. $\Pr[G_4^A \Rightarrow 1] = \Pr[G_5^A \Rightarrow 1]$.

Proof. We must show that the two games induce the same distribution. The random-oracle output c is identical in both games. In G_4 the signing oracle returns $(s_1, s_2) \sim D_{\Lambda(\mathbf{B}), s, (0, c)}$; since $\Lambda(\mathbf{B})$ is the NTRU lattice associated with h and q , and the distribution is shifted by $(0, c)$, this is exactly $D_{\mathcal{R}, s}$ conditioned on $s_2 - h s_1 \equiv c \pmod{q}$. In G_5 the output is again drawn from $D_{\mathcal{R}, s}$, and the condition $s_2 - h s_1 \equiv c \pmod{q}$ holds by construction.

Reduction from $\mathcal{R}$ -ISIS. We can now reduce $\mathcal{R}$ -ISIS to Game G_5 .

Claim. There exists an adversary B against $\mathcal{R}$ -ISIS such that

$$\Pr[G_5^A \Rightarrow 1] \leq \text{Adv}_{1,q,\alpha,\beta,B}^{(Q_H+1)\text{-}\mathcal{R}\text{-ISIS}}.$$

Proof. The adversary B is formally specified in Fig. 6. Thanks to the previous game hops, B can perfectly simulate G_5 for A without knowing the secret key for h . Suppose that A wins G_5 , i.e. the forgery verifies and $(m^*, \cdot)$ was never queried before. The challenge random-oracle output c^* then either matches one of B 's targets $\hat{c}_{i^*}$ or coincides with a signing-oracle query; the latter case is excluded by the freshness condition on $(m^*, \cdot)$, so there must exist some i^* with $\hat{c}_{i^*} = c^*$. The winning condition for B is therefore $s_2 - hs_1 \equiv \hat{c} \pmod{q}$, and the norm bound for A directly yields $\|(s_1^*, s_2^*)\| \leq \beta$, completing the proof. $\square$

Fig. 6: Adversary B against t - $\mathcal{R}$ -ISIS.

$B(h, \hat{c}_1, \dots, \hat{c}_{Q_H+1})$	Oracle $H(h, r, M)$
1: $\mathcal{H}, \mathcal{Q} \leftarrow \emptyset$	1: if $\exists c$ s.t. $(c, h, r, M) \in \mathcal{H}$
2: $\text{cnt}, l \leftarrow 0$	2: return c
3: $(M^*, \text{sig}^*) \leftarrow A^{\text{Sign}(\cdot), H(\cdot, \cdot, \cdot)}(h)$	3: $l \leftarrow l + 1$
4: parse $\sigma^* \rightarrow (r^*, s_1^*)$	4: $c \leftarrow \hat{c}_l$
5: $c^* \leftarrow H(h, r^*, M^*)$	5: $\mathcal{H} \leftarrow \mathcal{H} \cup \{(c, h, r, M)\}$
6: $s_2^* \leftarrow c^* + h \cdot s_1^* \pmod{q}$	6: return c
7: find i^* s.t. $c^* = \hat{c}_{i^*}$	Oracle $H'(h, r, M)$
8: return (i^*, s_2^*, s_1^*)	1: return $G_5.H'(h, r, M)$
Oracle $\text{Sign}(M)$	
1: return $G_5.\text{Sign}(M)$	

Table 4: Concrete security of YUANYANG.DSA for plain unforgeability.

	YUANYANG.DSA-512	YUANYANG.DSA-1024	YUANYANG.DSA-2048
Degree of $\mathcal{R}$	512	1024	2048
Bit security for t - $\mathcal{R}$ -ISIS	129	293	653
Max signing queries, Q_s	2^{80}	2^{80}	2^{80}
Max repetitions, C_s	$2^{80} + 2^{77}$	$2^{80} + 2^{73}$	$2^{80} + 2^{66}$
Rényi order, a_p	21592.9	1908.6	168.7
Rényi order, a_u	103.02	120.04	13.76
Bits lost from $r_p^{C_s}, \text{loss}_p$	0.06	0.69	7.75
Bits lost from $r_u^{C_s}, \text{loss}_u$	3.8	22	128
Final bit security	124	270	515

5.2 From provable security to concrete parameters

The reduction above provides qualitative assurance that breaking YUANYANG.DSA is at least as hard as solving t - $\mathcal{R}$ -ISIS, and hence rests on well-studied lattice assumptions. As is now standard practice for lattice-based signatures and KEMs — including the NIST standards ML-DSA, ML-KEM, and FN-DSA — we do *not* use the reduction itself to drive the choice of parameters. Provable bounds in this regime tend to be loose, and tracking them strictly would force unnecessarily large parameters. Instead, we follow the now-standard methodology of selecting parameters so that the cost of the best known cryptanalytic attacks

comfortably exceeds the targeted security level. The remainder of this section is therefore devoted to estimating that cost: we describe the model used for lattice reduction, then analyse the two attack avenues — key recovery and signature forgery — that determine our parameter choices.

5.2.1 Model for lattice reduction.

Throughout, we adopt the *Geometric Series Assumption* (GSA), which states that the Gram–Schmidt norms of a reduced basis decay geometrically. Concretely, for the self-dual BKZ (DBKZ) reduction algorithm of Micciancio and Walter [MW17], the output basis $(\mathbf{b}_1, \dots, \mathbf{b}_n)$ produced with block size β on a lattice $\mathcal{L}$ of rank n satisfies

$$\|\mathbf{b}_i^*\| = \delta_\beta^{n-2(i-1)} \det(\mathcal{L})^{1/n}, \quad \text{where } \delta_\beta = \left(\frac{(\pi\beta)^{1/\beta} \cdot \beta}{2\pi e} \right)^{1/2(\beta-1)},$$

and $\mathbf{b}_i^*$ denotes the i -th Gram–Schmidt vector of the basis.

5.2.2 Key-recovery attack.

The key-recovery problem consists in retrieving the secret key $(f, g) \in \mathcal{R}^2$ from the public data q and h alone. To the best of our knowledge, the strongest attacks proceed via lattice reduction. They build the algebraic lattice over $\mathcal{R}$ spanned by $(1, h)$ and $(0, q)$, namely the public NTRU basis, and search for the lattice vector $\mathbf{s} = (\mathbf{f}, \mathbf{g})$ (or any functionally equivalent vector such as $(\mu f, \mu g)$ for a unit μ of the number field). We write $\|\mathbf{s}\| = \sqrt{2d} \sigma_{\text{key}}$ for the resulting effective coefficient width.

To avoid enumerating the full sphere of radius $\sqrt{2d} \sigma_{\text{key}}$ — which, by the Gaussian heuristic, contains roughly $(2d\sigma_{\text{key}}^2/q)^d$ vectors — we rely on the so-called *projection trick*. Concretely, fix a block size β and run DBKZ on the public basis, yielding $[\mathbf{b}_1, \dots, \mathbf{b}_{2d}]$. If the *projection* of the secret key onto $\mathcal{P}$, the orthogonal complement of $\text{Span}(\mathbf{b}_1, \dots, \mathbf{b}_{2d-\beta-1})$, can be recovered, then the full secret key is obtained in polynomial time via Babai’s nearest-plane algorithm. It therefore suffices to find the projected secret key among the shortest vectors of the lattice generated by the last β basis vectors projected onto $\mathcal{P}$. Sieving on this projected lattice classically recovers all vectors of norm at most $\sqrt{4/3} \ell$, where ℓ is the norm of the $(2d - \beta)$ -th Gram–Schmidt vector $\mathbf{b}_{2d-\beta}^*$ of the reduced basis. Under the GSA,

$$\ell = \sqrt{q} \delta_\beta^{-2d+2\beta+2} \approx \sqrt{q} \left(\frac{\beta}{2\pi e} \right)^{1-d/\beta}.$$

Modelling $\mathbf{s}$ as a random vector of norm $\sqrt{2d} \sigma_{\text{key}}$ and using the GSA to bound the Gram–Schmidt norms of $\mathbf{b}_1^*, \dots, \mathbf{b}_{2d-\beta}^*$, the projection of $\mathbf{s}$ onto $\mathcal{P}$ has norm roughly

$$\sqrt{\frac{\beta}{2d}} \|\mathbf{s}\| = \beta^{1/2} \sigma_{\text{key}}.$$

The projection is therefore recovered by sieving as soon as $\beta^{1/2} \sigma_{\text{key}} \leq \sqrt{4/3} \ell$, i.e. when

$$\sigma_{\text{key}}^2 \leq \frac{4q}{3\beta} \delta_\beta^{4(\beta+1-d)}. \quad (7)$$

5.2.3 Signature forgery by reduction to Approx-CVP.

In a hash-and-sign signature, producing a forgery amounts to finding a lattice point $\mathbf{v}$ at bounded distance from a random target $\mathbf{x}$ in the ambient space. This Approx-CVP problem can be tackled with the *Nearest-Cospace* framework of [EK20]. Under the GSA, Theorem 3.3 of [EK20] guarantees that, whenever $\|\mathbf{x} - \mathbf{v}\| \leq \delta_\beta^{2d} q^{1/2}$, decoding can be performed in $\text{Poly}(d)$ calls to a CVP oracle in dimension β .

As noted in [CPS⁺20], a standard refinement of this attack restricts the decoding to the sublattice spanned by a subset of the public basis vectors. The only useful subset turns out to be obtained by dropping the first $k \leq n$ vectors: the working dimension shrinks by k , at the price of inflating the covolume by a factor $q^{k/(2(2d-k))}$. The decoding condition then becomes the slightly more general inequality $\|\mathbf{x} - \mathbf{v}\| \leq \min_{k \leq d} \delta_\beta^{2d-k} q^{d/(2d-k)}$, so that we need

$$\gamma \geq \min_{k \leq d} \delta_\beta^{2d-k} q^{d/(2d-k)}. \quad (8)$$

5.2.4 Concrete security.

We assess the concrete security of YUANYANG.DSA by following the standard cryptanalytic methodology: we estimate the cost of the best known attacks against *key recovery* and against *signature forgery*.

These cost estimates are converted into concrete bit-security values via the methodology of NEWHOPE [ADPS16]. This approach is often called the “core-SVP methodology”. In this model [BDGL16, Laa16], the bit complexity of lattice sieving — asymptotically the best SVP oracle — in dimension β is set to $\lceil 0.292\beta \rceil$ in the classical setting and $\lceil 0.265\beta \rceil$ in the quantum setting.

Table 5: Security level for YUANYANG.DSA
(C is classical security, Q is quantum security.)

	YUANYANG.DSA–512	YUANYANG.DSA–1024	YUANYANG.DSA–2048
Key recovery (C/Q)	128/116	266/242	545/494
Sig forgery (C/Q)	128/116	293/266	651/591

5.3 Other attacks on YUANYANG.DSA

We now briefly review additional families of attacks on the signature that turn out to be irrelevant for our parameter sets.

5.3.1 Algebraic attacks

As is well known from the design of NTRU-based schemes (e.g. FALCON or ModFalcon), the convolution ring $\mathcal{R}$ underlying YUANYANG.DSA carries a rich algebraic structure. Nevertheless, no known technique exploits this structure to beat its general-lattice counterpart by more than polynomial factors (see, for instance, the lattice-reduction speed-up of [KEF20]).

5.3.2 Overstretched NTRU regime

As observed in [KF17], when the modulus q is much larger than the magnitudes of the NTRU secret-key coefficients, lattice-reduction-based key-recovery attacks become significantly more effective than the analysis above suggests. This “overstretched NTRU” regime is reached when $q > (2d)^{2.83}$ for binary secrets; as is the case for FALCON and other NTRU-based NIST candidates, our parameters lie far from this regime, so even very significant improvements to this attack would not threaten YUANYANG.DSA.

5.3.3 Hybrid attacks

Odlyzko’s meet-in-the-middle attack and, more recently, Howgrave-Graham’s hybrid attack [How07] — which combines meet-in-the-middle with lattice-reduction-based key recovery — were effective against NTRU largely because the original NTRU secrets were sparse. In YUANYANG.DSA the secrets are dense elements of $\mathcal{R}$, so these attacks have no meaningful impact on parameter selection.

5.4 Joint Security

The defining feature of YuanYang is that the KEM and the signature are not two unrelated primitives placed next to each other. They are deployed with one NTRU public key and one shared NTRU secret core. This section records the resulting joint-security guarantee. The goal is not to repeat the standalone proofs: the UF-CMA proof of YUANYANG.DSA is given above, while the IND-CCA proof of YUANYANG.KEM is given in the companion KEM specification. Instead, we make explicit why opening both interfaces under the same key does not create an additional attack surface beyond the losses already accounted for in the two standalone analyses.

For a concrete YuanYang parameter set, a joint key pair is generated once. The public key is the NTRU element

$$\text{pk} = h = g/f \bmod q \in \mathcal{R}_q.$$

The shared secret core contains the short NTRU material used by both algorithms. On the signature side, this material is completed into the NTRU trapdoor basis and accompanied by the sampler precomputation needed by YUANYANG.DSA. On the KEM side, it is accompanied by the inverse modulo the masking ideal and the FO implicit-rejection state needed by YUANYANG.KEM. These auxiliary values are component-specific caches or rejection states. They do not constitute a second independently generated NTRU key.

Joint KEM security. The joint IND-CCA experiment is the ordinary IND-CCA experiment for the KEM component, except that the adversary is additionally given access to a signing oracle for the DSA component under the same public key h . More precisely, after joint key generation, the challenger gives h to the adversary, answers signing queries $M \mapsto \text{Sign}(\text{sk}_{\text{DSA}}, M)$, and answers decapsulation queries $C \mapsto \text{Decaps}(\text{sk}_{\text{KEM}}, C)$. The challenge is a usual KEM challenge: the challenger computes an honest encapsulation (C^*, K_0) , samples an independent key K_1 from the key space, gives (C^*, K_b) for a hidden bit b , and refuses the decapsulation query C^* . The adversary wins if it outputs $b' = b$. We write

$$\text{Adv}_{\text{YY.KEM}}^{\text{J-IND-CCA}}(\mathcal{A}) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

This is the security notion relevant when an attacker can see signatures and mount chosen-ciphertext attacks at the same time.

Joint signature security. The joint UF-CMA experiment is the ordinary UF-CMA experiment for the DSA component, except that the adversary is additionally given access to the KEM decapsulation oracle under the same public key h . The challenger answers signing queries and decapsulation queries. The adversary wins if it outputs a pair (M^*, σ^*) such that M^* was not previously submitted to the signing oracle and

$$\text{Verif}(h, M^*, \sigma^*) = 1.$$

We denote this winning probability by

$$\text{Adv}_{\text{YY.DSA}}^{\text{J-UF-CMA}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ wins the joint UF-CMA experiment}].$$

YuanYang is jointly secure if both advantages are negligible for every efficient adversary.

Theorem 5.3 (Joint security of YuanYang). *Assume that all hash and XOF calls are domain-separated as specified in the Specifications section, and that the decapsulation implementation reveals only the final shared key and no intermediate decoding, correction, or failure information. Then, for every efficient joint adversary $\mathcal{A}$, there exist adversaries $\mathcal{B}_{\text{KEM}}$ against the standalone IND-CCA security of YUANYANG.KEM and $\mathcal{B}_{\text{DSA}}$ against the standalone UF-CMA security of YUANYANG.DSA, with comparable running time and query complexity, such that*

$$\text{Adv}_{\text{YY.KEM}}^{\text{J-IND-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{YY.KEM}}^{\text{IND-CCA}}(\mathcal{B}_{\text{KEM}}) + \varepsilon_{\text{sig-sim}},$$

and

$$\text{Adv}_{\text{YY.DSA}}^{\text{J-UF-CMA}}(\mathcal{A}) \leq \text{Adv}_{\text{YY.DSA}}^{\text{UF-CMA}}(\mathcal{B}_{\text{DSA}}) + \varepsilon_{\text{fo-sim}}.$$

Here $\varepsilon_{\text{sig-sim}}$ is exactly the signing-simulation loss already bounded in the proof of YUANYANG.DSA: the rejection-budget term, the salt-collision term, and the Rényi losses for replacing real trapdoor sampling by the simulated hash-and-sign distribution. Likewise, $\varepsilon_{\text{fo-sim}}$ is the decapsulation-simulation loss already bounded in the FO analysis of YUANYANG.KEM: the random-oracle terms and the contribution of the decryption-error probability. Consequently, combining the standalone bounds gives joint security under the same NTRU, Ring-LWE, and Ring-ISIS assumptions used in the preceding sections.

Proof (Proof overview). We explain the two reductions at the level needed for a specification document. The full game transitions are the same as in the standalone proofs; the only additional point is that the two oracle interfaces are now available under the same h .

First consider the joint IND-CCA experiment for the KEM. The adversary receives a signing oracle in addition to the usual KEM challenge and decapsulation oracle. By the hash-and-sign simulation used in

the proof of YUANYANG.DSA, real signatures can be replaced by simulated signatures whose distribution is indistinguishable from the real one up to the loss $\epsilon_{\text{sig-sim}}$. This replacement is possible without knowing the signing trapdoor, because the random-oracle output in the DSA domain is programmed together with a short preimage distributed as in the signature proof. Domain separation is the reason this programming does not alter the random oracles used by the KEM transform. After this replacement, the signing oracle is independent of the KEM challenge except through the public key h , and the remaining experiment is precisely the standalone IND-CCA experiment for YUANYANG.KEM. Any adversary distinguishing the challenge key can therefore be converted into $\mathcal{B}_{\text{KEM}}$, plus the signing-simulation loss.

Now consider the joint UF-CMA experiment for the signature. The extra oracle is KEM decapsulation. In the FO transform with implicit rejection, invalid ciphertexts are answered by a key derived from the fallback secret and the ciphertext, while valid ciphertexts are tied to the unique message recovered by deterministic re-encryption. The standalone KEM proof already shows how to simulate this interface up to the loss governed by random-oracle queries and the decryption-error probability. Since the KEM hash domains are disjoint from the DSA hash-to-target domain, this simulation does not interfere with the programming steps in the signature proof. After replacing the decapsulation oracle by its FO simulation, the adversary sees exactly the same signing interface as in the standalone UF-CMA experiment. A successful fresh forgery therefore yields the same Ring-ISIS solution as in the proof of YUANYANG.DSA, except for the FO-simulation loss $\epsilon_{\text{fo-sim}}$.

The theorem follows by combining these two hybrid arguments. The shared NTRU public key h is already the public input in both standalone analyses, so sharing it does not by itself add a new secret-dependent transcript. The component-specific auxiliary states are also harmless: the signing sampler cache only affects how signatures are produced, and the KEM fallback state only affects implicit rejection. Both effects are already covered by the corresponding standalone simulations. What must be enforced at implementation level is that the decapsulation routine returns only the final key and that the logical random-oracle domains remain separated by their fixed public labels.

Remark 5.4. In the algorithms above, we write the hash call simply as H in order to keep the description readable. In the standalone security analysis, H is modelled as a random oracle. For the joint-security statement in this section, we additionally require the random oracle domain used by YUANYANG.DSA to be separated from the hash/XOF domains used by the companion YUANYANG.KEM. Thus, when YuanYang is deployed in the key reuse setting, the implementation should realise the signing hash H and the KEM hash/XOF calls by the same underlying XOF only after prepending fixed public labels identifying their respective roles.

6 Performance Evaluation

This section therefore reports both the complete API cost and the internal cost distribution. Unless explicitly identified as a diagnostic comparison, all figures include the submission-provided hash, XOF, and deterministic random number generation software.

6.1 Implementation and test platform

Implementation method. The reference and performance-optimized versions are assessed separately for all three parameter sets. The optimized implementation uses separate x86 intrinsic translation units, 64-bit fixed-point/integer arithmetic for signing and verification, and native floating-point arithmetic only during key generation. Polynomial and matrix-vector products in the signer use negacyclic NTTs, with two-prime CRT reconstruction when one prime does not provide enough range. The optimized x86 build contains separate AVX2 translation units for the signing NTT. Support for these instructions is selected at compilation time. The implementation uses the ICCS-supplied SM3 and KDF-SM3 auxiliary functions and software SM3-DRNG for the submission measurements. It uses no external cryptographic library.

Hardware and operating system. Measurements were made on a Dell Precision 3591 with one Intel Core Ultra 7 155H processor (16 physical cores and 22 logical CPUs), 32GB RAM, 18MiB aggregate L2 cache, and 24MiB L3 cache. The operating system was 64-bit Ubuntu 24.04.4 LTS with Linux kernel 6.17.0-generic. Every measured process was pinned with `taskset -c 1` to logical CPU 1, a performance core; the benchmark itself is single-threaded.

The `intel_pstate` governor was set to `performance`; the resulting P-core policy reported a 4.80 GHz.

Compiler, flags, and libraries. The compiler was Ubuntu GCC 13.3.0; the build used GNU Make 4.3. Following the x86 self-assessment guideline, the reference version used

`-std=c99 -Wpedantic -Wall -Wextra -O2,`

and the performance-optimized version used

`-Ofast -march=x86-64 -mavx2 -mtune=native -flto
-fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra.`

Executables were linked only against the system C runtime and `libm` (`-lm`).

Measurement method. For each parameter set and implementation version, a warmed-up process performed 1000 key generations, signatures, and successful verifications. Every iteration generated a new key pair and signed the same reproducible 64-byte message 0001 ... 3f. The submission software DRNG was reinitialized from a documented deterministic seed for each key generation and signature. The tables give arithmetic means over all 1000 measurements. DRNG initialization was outside the timed region, while random-byte generation, key/signature encoding, and all rejection retries were included.

6.2 Complete API performance

Table 6 reports the complete API with the submission-provided software DRNG and auxiliary hash/XOF.

Table 6: Performance-optimized-version single-thread self-assessment. Values are averages over 1000 operations on an exact 64-byte message. Operation cells report average CPU time, average cycles, and throughput, in that order.

	$d = 512$	$d = 1024$	$d = 2048$
KeyGen	2.884 ms	4.593 ms	19.292 ms
	8.637 Mcy	13.759 Mcy	57.787 Mcy
	346.673 ops/s	217.632 ops/s	51.826 ops/s
Sign	1.289 ms	2.432 ms	5.780 ms
	3.862 Mcy	7.286 Mcy	17.315 Mcy
	775.241 ops/s	172.958 ops/s	64.203 ops/s
Verif	46.342 μ s	98.557 μ s	203.829 μ s
	138.190 Kcy	294.593 Kcy	609.935 Kcy
	21,485.527 ops/s	10,118.833 ops/s	4,898.101 ops/s
public key	738 bytes	1,570 bytes	3,330 bytes
private key	15,584 bytes	31,264 bytes	62,720 bytes
signature	561 bytes	1,150 bytes	2,364 bytes
static memory	2,048 KiB	2,176 KiB	2,156 KiB
peak memory	3,896 KiB	4,424 KiB	6,084 KiB

6.3 Correctness, resources, and storage overhead

All 1000 signatures in each measured instance verified successfully. Separately, the deterministic `kat_ngcc` workflow generated, replayed, and verified the ten checked-in KAT cases for every reference and optimized parameter set. The corresponding vectors are recorded under `API_PKC/Test_Vector`.

The key and signature sizes in Table 6 are the lengths returned by the API. Signature size includes the salt. Static memory is measured as resident memory after the executable and benchmark I/O are initialized but before the first cryptographic operation. Peak memory is the process maximum resident set across the measured key-generation, signature, and verification operations. These process-level values therefore include the executable, constants, globals, stack, heap, and linked runtime.

6.4 Cost of the submission auxiliary software

The auxiliary software has a material effect on these figures. In particular, the ICCS software DRNG is an SM3-based generator and is called repeatedly by the Gaussian samplers. To isolate this cost, we rebuilt the same source with the same compiler flags, but switched the software DRNG to the system-RNG. On Linux this selects the buffered `getrandom` backend. This comparison changes only the source of random bytes. It is a diagnostic, not the submission/KAT configuration. Each entry below is an average over 1000 operations with the same 64-byte message and single-core pinning as the formal optimized assessment.

Table 7: Matched software-DRNG versus operating-system-RNG comparison. Times are 1000-operation averages.

		$d = 512$	$d = 1024$	$d = 2048$
KeyGen	software DRNG (ms)	2.884	4.596	19.292
	system RNG (ms)	2.181	4.463	15.317
	software DRNG (ms)	1.289	2.432	5.780
Sign	system RNG (ms)	0.746	1.479	3.482

Thus, replacing the required software DRNG raises signing throughput by factors of 1.73, 1.64, and 1.66 respectively. This does not indicate an algorithmic difference: it measures the cost of generating the same type of sampler input through two different backends. The smaller and more variable key-generation difference is expected because PairGen and the NTRU solver, rather than random-byte expansion alone, dominate key generation.

Verification consumes no signing randomness, but it is strongly affected by the mandatory auxiliary hash/XOF. A fresh instrumented 1,000-verification run for YUANYANG.DSA-512 attributed 11.29% of verification time to SM3($m \parallel h$) and 48.43% to KDF-SM3 HashToChallenge, for 59.72% combined. Signature decompression accounted for 20.53%, recovery of $s_2 = c + hs_1$ for 14.21%, and the final squared-norm test for 1.03%. Consequently, the auxiliary XOF is the largest individual verification cost. These auxiliary functions are for correctness checking and preliminary performance evaluation and will be replaced in a later evaluation round; comparisons with implementations using a different hash/XOF backend are not like-for-like, which prevents meaningful comparison to raw numbers from other schemes like Falcon.

6.5 Internal performance distribution

Instrumentation adds timing calls and therefore its absolute totals are not substituted into Table 6; its purpose is to locate the expensive steps. In a 100-run YUANYANG.DSA-512 key-generation profile, NTRUSolve used 48.03% and PairGen 41.22% of the total time. The high cost of PairGen is mostly due to its high restart rate: on average, we need to re-sample f and g 35.6 times to pass the checks. Key assembly used 4.73%, the perturbation decomposition 2.93%, public-key construction 1.79%, and the SecurityLoss check 1.21%. There were 1.680 outer attempts per generated key on average; 68 of 168 candidates were rejected by SecurityLoss. Optimization of signing NTTs therefore has little effect on key generation, whose principal costs are trapdoor-pair generation and solving the NTRU equation.

In the corresponding 1,000-signature profile, lattice sampling occupied 85.29% of instrumented signing time. The largest nested components were ring sampling (49.76%), offline-capable presampling (24.08%), and the fixed-point theta correction (16.55%). Key-dependent NTT precomputation used 10.73%, and the NTT/CRT quadratic-form work for the δ_2 rejection test used 8.95%. These percentages are hierarchical and overlap: for example, presampling and δ_2 are contained in the lattice-sampling total. The current signing bottleneck is therefore the sampler and its rejection work, not a single remaining NTT butterfly.

6.6 Batch and online-offline signing

YUANYANG.DSA exposes a useful online-offline structure when many messages are signed with one key. The ordinary submission API remains stateless: every `sig_sign` call decodes the secret key, performs

the key-dependent NTT precomputation, and generates presamples on demand. The batch path instead decodes once, computes the NTT representation once per key, and fills a caller-owned ring buffer with message-independent presamples. The online phase then hashes the message, constructs the challenge, consumes one fresh presample for each sampling attempt, completes rejection sampling, and encodes the signature. This behavior would match one of a system service or a long-lived application, that will reuse the same key to sign, and can generate presamples on down-time.

The NTT precomputation is required to make the repeated matrix–vector arithmetic fast. It converts the key-dependent $\hat{A}$, $\hat{u}$, trapdoor-basis, $\hat{B}^{-1}$, and Σ_δ correction polynomials to one-prime or two-prime NTT representations and imports reusable operands to Montgomery form. Negacyclic polynomial matrix–vector products then become pointwise products plus only the transforms of the message- or sample-dependent operand; wide products are reconstructed with CRT. Without this per-key step, the same key polynomials would be transformed again for every product and signature.

The offline presampler depends only on the key, the NTT precomputation, and fresh randomness; it does not consume a message, salt, or challenge. Each stored presample is single-use. A refill occurs when the buffer drops below 128 entries (or below its capacity if smaller), and fills it back to the selected depth. More presamples can be generated than signatures because a signature may require several rejection attempts and because the final refill can leave unused entries.

Table 8 reports one 1,000-signature run with a 256-slot buffer, on the same pinned core and with the submission DRNG (timing should therefor be compared to Table 6). “Online” excludes all explicitly offline work. “Amortized” adds total presampling time divided by 1,000 to online time; it deliberately excludes the separately reported one-time key generation, key decode, and NTT precomputation.

Table 8: Depth-256 batch signing with the submission software DRNG.

	$d = 512$	$d = 1024$	$d = 2048$
one-time setup keygen (ms)	3.834	5.930	13.616
private-key decode (μ s)	13	29	49
NTT precomputation (μ s)	172	377	763
offline presample/slot (ms)	0.412	0.916	1.875
presamples generated	1,418	1,288	1,551
presamples consumed	1,181	1,076	1,304
full signature (ms)	1.222	2.374	6.044
online batch signature (ms)	0.559	1.052	2.672
full/online speedup	2.18×	2.26×	2.26×

The one-key setup values in Table 8 are diagnostic single measurements, not replacements for the 1000-operation formal averages in Table 6. The full-signature row also uses that one fixed key, which explains normal differences from the multiple-key API benchmark.

7 Conclusion

This document specifies the YUANYANG.DSA signature scheme: a lattice-based GPV hash-and-sign signature instantiated over NTRU lattices. As a general-purpose signature algorithm, YUANYANG.DSA has the following advantages in terms of security, efficiency and portability.

Small size. YUANYANG.DSA offers very small public keys and signatures among comparable lattice-based signatures. This compactness makes YUANYANG.DSA an attractive choice for large-scale post-quantum transitions, especially in bandwidth-limited applications, e.g. digital certificates, smart cards, and blockchains.

Strong security. YUANYANG.DSA admits a formal theoretical security proof based on well-studied assumptions, and its parameters achieve the target security level in a conservative manner. The GPV framework also

gives a clear security proof in the quantum random oracle model (QROM) [BDF⁺11]. In contrast, the QROM security of Fiat-Shamir lattice-based signatures would require either certain conditions [DFMS19, LZ19] or larger parameters [KLS18, JMW24].

Fast signing and verification. The signing and verification procedures are quite fast, significantly outperforming widely deployed RSA signatures. In particular, the signing algorithm is well suited for parallelization, allowing high-throughput implementations on GPUs, which is of special interest in cloud services. Additionally, verification is very fast and simple, which makes YUANYANG.DSA suitable for certain applications, e.g. vehicle-to-vehicle communications.

Good portability. YUANYANG.DSA avoids the delicate signing procedure and reliance on floating-point arithmetic of FALCON-type signatures. This facilitates implementation on various platforms, especially on constrained devices, and avoids potential risks.

The algorithmic structure of YUANYANG.DSA enables several specialized modes and optimizations facilitating certain applications.

Key-recovery and message-recovery modes. Like FALCON, YUANYANG.DSA can be instantiated in both key-recovery and message-recovery modes. The key-recovery mode compresses the public key into a short hash at the cost of doubling the signature size, which further reduces the combined public key and signature size. In the message-recovery mode [dPLP16], the signature is also twice as long, but it allows recovery of an even longer message, which also reduces the bandwidth.

Online-offline mode. Like MITAKA, YUANYANG.DSA has an online-offline structure in which most randomness-consuming Gaussian samplings can be performed in the offline phase. This further accelerates and simplifies the signing procedure.

In addition, YUANYANG.DSA offers some attractive features beyond a standalone signature scheme, making it a building block for versatile protocols.

Key reuse along with YUANYANG.KEM. As indicated by its name, YUANYANG.DSA has a paired KEM — YUANYANG.KEM. They can share the same key pair in a provably secure manner. This opens up promising avenues for designing more efficient protocols, e.g. chip-based payments [DLP⁺12] and end-to-end-encrypted messaging [FGPZ25].

Function extensions. YUANYANG.DSA can be converted into a (hierarchical) identity-based encryption (IBE) scheme [DLP14, ZMS⁺24]. In particular, the FALCON-based IBE [ZMS⁺24] is under consideration for NCSC and ETSI standardization, and YUANYANG.DSA can further improve this IBE scheme. Moreover, YUANYANG.DSA can also be adapted to aggregate signatures [AAB⁺24] and ring signatures [LAZ19, GJK24] by existing constructions.

References

- AAB⁺24. M. A. Aardal, D. F. Aranha, K. Boudgoust, S. Kolby, and A. Takahashi. Aggregating falcon signatures with LaBRADOR. In *CRYPTO 2024, Part I*, vol. 14920 of *LNCS*, pp. 71–106. Springer, Cham, 2024. 35
- ADPS16. E. Alkim, L. Ducas, T. Pöppelmann, and P. Schwabe. Post-quantum key exchange - A new hope. In *USENIX Security 2016*, pp. 327–343. USENIX Association, 2016. 29
- BDF⁺11. D. Boneh, Ö. Dagdelen, M. Fischlin, A. Lehmann, C. Schaffner, and M. Zhandry. Random oracles in a quantum world. In *ASIACRYPT 2011*, vol. 7073 of *LNCS*, pp. 41–69. Springer, Berlin, Heidelberg, 2011. 35
- BDGL16. A. Becker, L. Ducas, N. Gama, and T. Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In *27th SODA*, pp. 10–24. ACM-SIAM, 2016. 29
- CPS⁺20. C. Chuengsatiansup, T. Prest, D. Stehlé, A. Wallet, and K. Xagawa. ModFalcon: Compact signatures based on module-NTRU lattices. In *ASIACCS 20*, pp. 853–866. ACM Press, 2020. 28
- CS97. D. Coppersmith and A. Shamir. Lattice attacks on NTRU. In *EUROCRYPT’97*, vol. 1233 of *LNCS*, pp. 52–61. Springer, Berlin, Heidelberg, 1997. 3
- DEP23. L. Ducas, T. Espitau, and E. W. Postlethwaite. Finding short integer solutions when the modulus is small. In *CRYPTO 2023, Part III*, vol. 14083 of *LNCS*, pp. 150–176. Springer, Cham, 2023. 4
- DFMS19. J. Don, S. Fehr, C. Majenz, and C. Schaffner. Security of the Fiat-Shamir transformation in the quantum random-oracle model. In *CRYPTO 2019, Part II*, vol. 11693 of *LNCS*, pp. 356–383. Springer, Cham, 2019. 35
- DGPY20. L. Ducas, S. Galbraith, T. Prest, and Y. Yu. Integral matrix gram root and lattice gaussian sampling without floats. In *EUROCRYPT 2020, Part II*, vol. 12106 of *LNCS*, pp. 608–637. Springer, Cham, 2020. 4
- DLP⁺12. J. P. Degabriele, A. Lehmann, K. G. Paterson, N. P. Smart, and M. Strefer. On the joint security of encryption and signature in EMV. In *CT-RSA 2012*, vol. 7178 of *LNCS*, pp. 116–135. Springer, Berlin, Heidelberg, 2012. 5, 35
- DLP14. L. Ducas, V. Lyubashevsky, and T. Prest. Efficient identity-based encryption over NTRU lattices. In *ASIACRYPT 2014, Part II*, vol. 8874 of *LNCS*, pp. 22–41. Springer, Berlin, Heidelberg, 2014. 1, 35
- DP16. L. Ducas and T. Prest. Fast fourier orthogonalization. In *Proceedings of the ACM on International Symposium on Symbolic and Algebraic Computation, ISSAC 2016, Waterloo, ON, Canada, July 19–22, 2016*, pp. 191–198. ACM, 2016. 4
- dPLP16. R. del Pino, V. Lyubashevsky, and D. Pointcheval. The whole is less than the sum of its parts: Constructing more efficient lattice-based AKEs. In *SCN 16*, vol. 9841 of *LNCS*, pp. 273–291. Springer, Cham, 2016. 35
- EFG⁺22. T. Espitau, P.-A. Fouque, F. Gérard, M. Rossi, A. Takahashi, M. Tibouchi, A. Wallet, and Y. Yu. Mitaka: A simpler, parallelizable, maskable variant of falcon. In *EUROCRYPT 2022, Part III*, vol. 13277 of *LNCS*, pp. 222–253. Springer, Cham, 2022. 1, 3, 13
- EK20. T. Espitau and P. Kirchner. The nearest-colattice algorithm: Time-approximation tradeoff for approx-cvp. *Open Book Series*, 4(1):251–266, 2020. 28
- ENS⁺23. T. Espitau, T. T. Q. Nguyen, C. Sun, M. Tibouchi, and A. Wallet. Antrag: Annular NTRU trapdoor generation - making mitaka as secure as falcon. In *ASIACRYPT 2023, Part VII*, vol. 14444 of *LNCS*, pp. 3–36. Springer, Singapore, 2023. 1, 3, 4, 6, 7, 13
- ETWY22. T. Espitau, M. Tibouchi, A. Wallet, and Y. Yu. Shorter hash-and-sign lattice-based signatures. *IACR Cryptol. ePrint Arch., to appear at CRYPTO 2022*, p. 785, 2022. 12
- FGPZ25. R. Fiedler, F. Günther, J. Pan, and R. Zeng. XHMQV: Better efficiency and stronger security for signal’s initial handshake based on HMQV. In *CRYPTO 2025, Part VIII*, vol. 16007 of *LNCS*, pp. 109–140. Springer, Cham, 2025. 5, 35
- GJK24. P. Gajland, J. Janneck, and E. Kiltz. Ring signatures for deniable AKEM: Gandalf’s fellowship. In *CRYPTO 2024, Part I*, vol. 14920 of *LNCS*, pp. 305–338. Springer, Cham, 2024. 35
- GPV08. C. Gentry, C. Peikert, and V. Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *40th ACM STOC*, pp. 197–206. ACM Press, 2008. 1, 3
- GS02. C. Gentry and M. Szydło. Cryptanalysis of the revised NTRU signature scheme. In *EUROCRYPT 2002*, vol. 2332 of *LNCS*, pp. 299–320. Springer, Berlin, Heidelberg, 2002. 3
- How07. N. Howgrave-Graham. A hybrid lattice-reduction and meet-in-the-middle attack against NTRU. In *CRYPTO 2007*, vol. 4622 of *LNCS*, pp. 150–169. Springer, Berlin, Heidelberg, 2007. 29
- HPRR20. J. Howe, T. Prest, T. Ricosset, and M. Rossi. Isochronous gaussian sampling: From inception to implementation. In *PQCrypto 2020*, pp. 53–71. Springer, Cham, 2020.
- JMW24. K. A. Jackson, C. A. Miller, and D. Wang. Evaluating the security of CRYSTALS-Dilithium in the quantum random oracle model. In *EUROCRYPT 2024, Part VI*, vol. 14656 of *LNCS*, pp. 418–446. Springer, Cham, 2024. 35
- JZYT26. H. Jia, S. Zhang, Y. Yu, and C. Tang. Revisiting the Concrete Security of Falcon-type Signatures. In *PKC 2026*, 2026. 1, 3, 4, 5, 6, 8, 9
- KEF20. P. Kirchner, T. Espitau, and P.-A. Fouque. Fast reduction of algebraic lattices over cyclotomic fields. In *CRYPTO 2020, Part II*, vol. 12171 of *LNCS*, pp. 155–185. Springer, Cham, 2020. 29
- KF17. P. Kirchner and P.-A. Fouque. Revisiting lattice attacks on overstretched NTRU parameters. In *EUROCRYPT 2017, Part I*, vol. 10210 of *LNCS*, pp. 3–26. Springer, Cham, 2017. 29

- Kle00. P. N. Klein. Finding the closest lattice vector when it's unusually close. In *Proceedings of the Eleventh Annual ACM-SIAM Symposium on Discrete Algorithms, January 9-11, 2000, San Francisco, CA, USA*, pp. 937–941. ACM/SIAM, 2000.
- KLS18. E. Kiltz, V. Lyubashevsky, and C. Schaffner. A concrete treatment of Fiat-Shamir signatures in the quantum random-oracle model. In *EUROCRYPT 2018, Part III*, vol. 10822 of *LNCS*, pp. 552–586. Springer, Cham, 2018. 35
- KTW⁺24. K. Kim, M. Tibouchi, A. Wallet, T. Espitau, Y. Yu, and Y. Kim. Solmae: Faster and simpler quantum-safe signature based on ntru-lattices. *Technical Report, Ver. 2.0.*, 2024. Available at: <https://solmae-sign.info/>. 3
- Laa16. T. Laarhoven. *Search problems in cryptography: from fingerprinting to lattice sieving*. PhD thesis, Mathematics and Computer Science, 2016. Proefschrift. 29
- LAZ19. X. Lu, M. H. Au, and Z. Zhang. Raptor: A practical lattice-based (linkable) ring signature. In *ACNS 2019*, vol. 11464 of *LNCS*, pp. 110–130. Springer, Cham, 2019. 35
- LZ19. Q. Liu and M. Zhandry. Revisiting post-quantum Fiat-Shamir. In *CRYPTO 2019, Part II*, vol. 11693 of *LNCS*, pp. 326–355. Springer, Cham, 2019. 35
- MW17. D. Micciancio and M. Walter. Gaussian sampling over the integers: Efficient, generic, constant-time. In *CRYPTO 2017, Part II*, vol. 10402 of *LNCS*, pp. 455–485. Springer, Cham, 2017. 28
- Pei10. C. Peikert. An efficient and parallel Gaussian sampler for lattices. In *CRYPTO 2010*, vol. 6223 of *LNCS*, pp. 80–97. Springer, Berlin, Heidelberg, 2010.
- PFH⁺20. T. Prest, P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang. FALCON. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- PFH⁺22. T. Prest, P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang. FALCON. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>. 1, 17
- PP19. T. Pornin and T. Prest. More efficient algorithms for the NTRU key generation using the field norm. In *PKC 2019, Part II*, vol. 11443 of *LNCS*, pp. 504–533. Springer, Cham, 2019. 4, 6, 9
- Pre15. T. Prest. *Gaussian Sampling in Lattice-Based Cryptography*. PhD thesis, École Normale Supérieure, Paris, France, 2015. 1, 13
- ZJR⁺25. S. Zhang, H. Jia, D. Ran, Y. Yu, Y. Yu, and X. Wang. GPV preimage sampling with weak smoothness and its applications to lattice signatures. Cryptology ePrint Archive, Report 2025/1940, 2025. 1, 3, 4, 5, 11, 12, 20
- ZMS⁺24. R. K. Zhao, S. McCarthy, R. Steinfeld, A. Sakzad, and M. O'Neill. Quantum-safe HIBE: does it cost a latte? *IEEE Trans. Inf. Forensics Secur.*, 19:2680–2695, 2024. 35
- ZSS20. R. K. Zhao, R. Steinfeld, and A. Sakzad. Facct: Fast, compact, and constant-time discrete gaussian sampler over integers. *IEEE Transactions on Computers*, 69(1):126–137, 2020.